



Contents lists available at ScienceDirect

Journal of Information and Intelligence

journal homepage: www.keaipublishing.com/en/journals/journal-of-information-and-intelligence

An efficient self attention-based 1D-CNN-LSTM network for IoT attack detection and identification using network traffic

Tinshu Sasi^a, Arash Habibi Lashkari^{a,b}, Rongxing Lu^{a,*}, Pulei Xiong^c, Shahrear Iqbal^c^a Canadian Institute for Cybersecurity (CIC), Faculty of Computer Science, University of New Brunswick (UNB), Fredericton E3B 5A3, Canada^b Behaviour-Centric Cybersecurity Center (BCCC), School of Information Technology, York University, Toronto M3J 1P3, Canada^c National Research Council (NRC), Ottawa K1A 0R6, Canada

ARTICLE INFO

Keywords:

IoT attacks
Machine learning
Artificial intelligence
IoT attack detection

ABSTRACT

In the last ten years, the IoT has played a crucial role in society's digital transformation. However, because of the wide range of devices it encompasses, it is also facing increased security vulnerabilities. This research presents a novel mechanism called the self-attention-based 1D-CNN-LSTM, which uses convolutional neural networks (CNNs) combined with a long short-term memory (LSTM) model enhanced with a self-attention mechanism for detecting IoT attacks. The proposed mechanism achieves high accuracy and efficiently differentiates malicious and benign network traffic. By employing Shapley additive explanations (SHAP), we identified important predictive features from the preprocessed data, which were retrieved using CICFlowMeter. This has strengthened the dependability of the model. In addition, we enhanced the model by training it on a smaller collection of features, resulting in shorter training time while preserving accuracy. We have also generated nine augmented IoT tabular datasets named CIC-BCCC-NRC_TabularIoTAttack-2024 from accessible IoT datasets to evaluate the model's robustness and showcase its efficacy in IoT security.

1. Introduction

The digital revolution has substantially transformed our lives, with the Internet of Things (IoT) playing a prominent role. However, the rapid development of IoT in various aspects of life has led to numerous emerging cybersecurity threats. Consequently, detecting and preventing potential attacks in IoT networks has recently attracted significant interest from academia and industry. Among the various attack detection approaches, machine learning-based methods, especially deep learning, have demonstrated outstanding potential due to their early detection capabilities [1].

Over the past several years, there has been a substantial increase in the number of attacks specifically targeting IoT devices. These attacks encompass various types of cyber threats, such as infiltrating wireless webcams to gain unauthorized access to surveillance cameras and violate user privacy; targeting implantable cardiac devices to potentially deplete battery life, disrupt pacing, and cause electric shocks, thereby endangering patients' lives; compromising children's smartwatches to expose their location data and personal

* Corresponding author.

E-mail addresses: tinshu.sasi@unb.ca (T. Sasi), ahabil@yorku.ca (A.H. Lashkari), rlu1@unb.ca (R. Lu), Pulei.Xiong@nrc-cnrc.gc.ca (P. Xiong), Shahrear.Iqbal@nrc-cnrc.gc.ca (S. Iqbal).<https://doi.org/10.1016/j.jiixd.2024.09.001>

Received 12 June 2024; Received in revised form 28 August 2024; Accepted 9 September 2024

Available online 26 September 2024

2949-7159/© 2024 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

information, posing multiple safety risks; and manipulating the controller area network (CAN) bus of vehicles to potentially alter speed and direction, thereby posing a threat to public safety. An investigation into smart home hacking revealed that, over one week, fraudsters and unidentified groups launched over 12,000 attacks against various smart home devices, including TVs, thermostats, smart kettles, and security systems [2,3].

1.1. What are IoT attacks?

IoT attacks constitute cyber-attacks leveraging IoT devices to access consumers' sensitive data. Typically, attackers deploy malware on these devices, causing damage or infiltrating additional organizational data. Due to insufficiently designed security mechanisms, IoT devices emerge as prominent vulnerabilities within organizational infrastructures, posing substantial security risks. Basic IoT devices often lack robust built-in security measures to counter cyber threats. Given their limited functionalities and purposes, security considerations for such devices are frequently overlooked, rendering them susceptible to cyber-attacks. Hackers and organizations can exploit common flaws and “zero-day exploits” to attack IoT devices in various ways [4].

1.2. Motivation

The primary motivation for addressing the IoT attack detection problem lies in the critical need to protect IoT systems and networks from potential security vulnerabilities. As IoT devices become increasingly pervasive across various aspects of daily life, they are increasingly targeted by cyber attacks. The capability to accurately identify and differentiate between malicious and benign instances within IoT network traffic is vital for maintaining the integrity and reliability of these interconnected devices and services. Given the escalating complexity and volume of IoT network data, traditional security measures are often inadequate. Therefore, developing robust detection methodologies is essential for enhancing the security of IoT systems, preventing potential breaches, and ensuring the continued trust and functionality of IoT technologies in our daily lives.

The problem of IoT attack detection by identifying and differentiating between malicious and benign instances is multifaceted and encompasses several key challenges:

- **Feature selection:** Identifying the optimal features that can accurately distinguish between malicious and benign instances is critical. This involves analyzing a wide range of features and determining those most indicative of malicious behaviour.
- **Efficacy and reliability:** Establishing the efficacy and reliability of these features for detection purposes is a significant challenge. This requires rigorous testing and validation to ensure that the selected features consistently and accurately identify malicious activity, minimizing false positives and negatives.
- **Methodology development:** Developing robust detection methodologies capable of handling the escalating complexity and volume of IoT network data is imperative. These methodologies must be adept at processing large amounts of data in real-time and adapting to new and evolving threats.
- **System integration:** Ensuring the seamless integration of the developed methodologies into existing IoT systems and networks without causing disruptions or performance issues is also a crucial consideration.

By addressing these challenges, we aim to enhance the security and reliability of IoT systems, making them more resilient to potential cyber threats. This will safeguard the interconnected devices that play an increasingly central role in our lives.

To address the multifaceted problem of identifying and differentiating between malicious and benign instances in IoT network traffic, specifically focusing on feature selection, efficacy and reliability, and methodology development, we propose a self-attention-based 1D-CNN-LSTM network. This network leverages convolutional neural networks (CNNs), residual networks, and long short-term memory (LSTM) networks, in conjunction with attention mechanisms, to enhance the detection of IoT attacks in TCP network flows. The proposed approach capitalizes on the robust feature extraction capabilities of CNNs, the effective sequence learning abilities of LSTMs, and the precise contextual insights provided by self-attention mechanisms, resulting in a comprehensive and effective framework.

In summary, this paper makes the following contributions:

- We introduce a novel self-attention-based 1D-CNN-LSTM network to detect IoT attacks by analyzing time-related features extracted from TCP flow data.
- To ensure the robustness of our model, we evaluate its performance on nine publicly available external datasets, which have been enhanced and pre-processed using CICFlowMeter.
- We aim to identify the optimal combination of hyperparameters that yield the highest performance metrics for our model.
- We employ Shapley additive explanations (SHAP) to calculate feature importance scores, which are then used to identify the most significant features from the extracted feature set.
- By retraining the model on the most significant features, we reduce the model size while maintaining performance comparable to the original model.
- We release the CIC-BCCC-NRC_TabularIoTAttack-2024 dataset, which includes over 80 extracted features from nine IoT datasets.

The subsequent sections of this paper are structured as follows. In Section 2, foundational insights into the subject matter are provided, elucidating key technical terminologies pertinent to IoT attacks. This section also encapsulates an overview of the IoT attacks referenced in the training datasets. Section 3 delves into a comprehensive review of prior scholarly works on IoT attacks, exploring the

methodologies employed for detection utilizing machine learning and deep learning techniques. Furthermore, it presents an exhaustive classification of various types of IoT attacks. The rationale behind the proposed methodology is expounded upon in Section 4, elucidating the preprocessing steps and detailing the architectural framework, with a comprehensive breakdown of each constituent component. Section 5 offers an in-depth analysis of the research outcomes. This segment encompasses discussions on the dataset, experimental setup, features utilized, evaluation metrics employed, and the results garnered from the proposed methodology. Finally, Section 6 serves as the concluding segment of the paper, encapsulating a summary of the contributions made, addressing encountered challenges, and providing insights into potential avenues for future research endeavors.

2. Preliminaries

Before delving deeper into the intricacies of IoT attacks, acquiring a fundamental understanding of the subject is imperative. It is essential to thoroughly comprehend the various technical terminologies associated with this study area [4].

2.1. IoT architecture

The IoT architecture refers to the organization and setup of IoT devices to fulfil users’ particular requirements and requests. An IoT system is divided into three to seven layers, depending on their complexity, and each layer has a specific function. The lack of established protocols in the architecture of the IoT presents further difficulties regarding interoperability, security, and several other issues. The IoT architecture can include up to seven levels [5]. Fig. 1 provides a detailed illustration of the IoT and Industrial Internet of Things (IIoT).

- *Perception layer*: The perception layer, also known as the device layer, includes a variety of sensors such as RFID scanners, security cameras, GPS modules, and so on. These devices, such as conveyor systems, industrial robots, and automated guided vehicles (AGVs), may be used with industrial gear. These gadgets collect sensory data, monitor the production floor and surroundings, transport raw materials, and so forth [6].
- *Transport layer/network layer*: The transport/network layer is responsible for transferring data to the processing systems of the subsequent layer [6]. IoT gateways must first transform the incoming input from analogue to digital format. Subsequently, the gateway can transmit the data to a local or cloud data center via several data transfer protocols (DTPs). Some of the leading IoT protocols are Bluetooth, Wi-Fi, Zigbee, Z-Wave, 6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks), MQTT

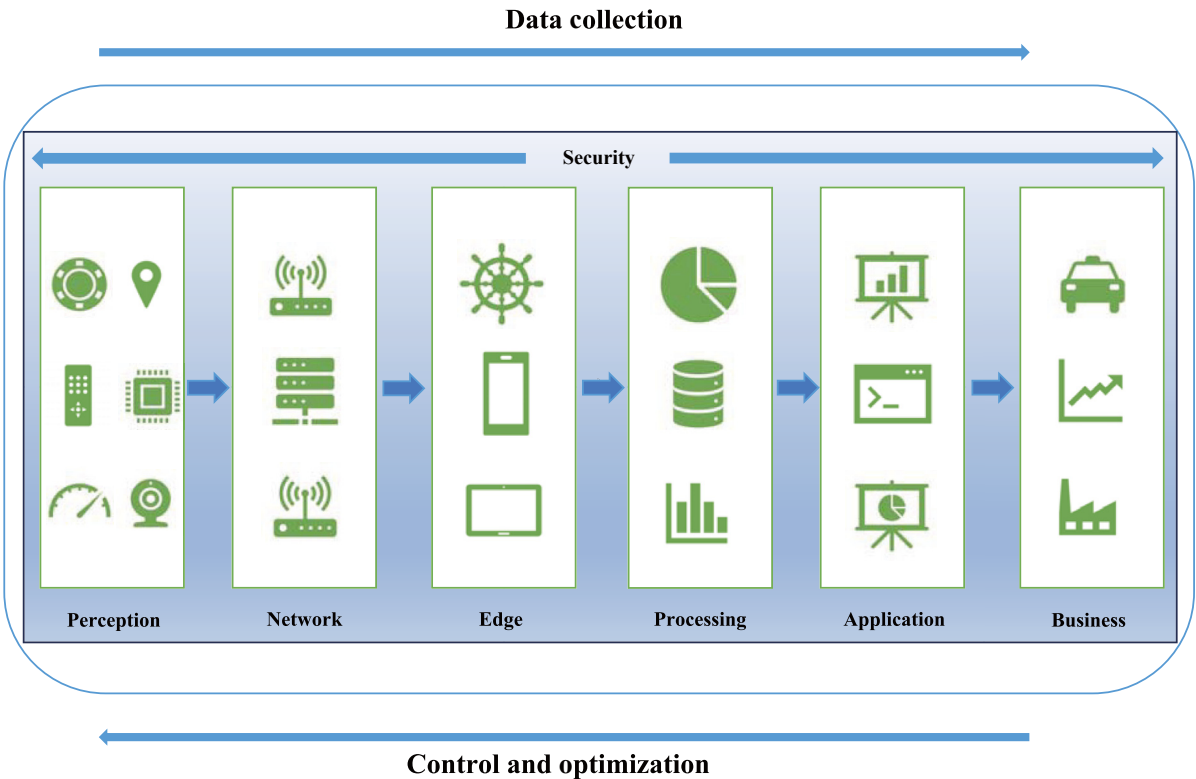


Fig. 1. IoT architecture levels.

(Message Queue Telemetry Transport), CoAP (Constrained Application Protocol), DDS (Data Distribution Service), AMQP (Advanced Message Queuing Protocol) [5].

- **Edge layer:** The edge layer in an IoT configuration consists of the physical hardware, embedded operating system, and device firmware. With the growing number of interconnected devices, latency becomes a prominent issue in more extensive IoT networks. Edge computing, aided by the edge layer, resolves this problem by allowing data processing and analysis close to the data source. Latency, the period of time between the detection of an event and the execution of an action, is a critical concern for devices that are linked to a network. Reducing latency may be accomplished by placing processing resources close to the sensors or at the network edge, enabling prompt connection and data exchange between devices [7].
- **Processing layer:** The processing layer, sometimes called the middleware layer, comprises servers and databases. Its primary functions include decision-making, executing optimization algorithms, and storing large amounts of data [6]. This layer consists of cloud computing platforms that can analyze and interpret data from the physical environment. The system processes unprocessed sensor data and transforms it into relevant insights using cloud services and comprehensive data modules. Furthermore, the processing layer allows the system to promptly respond to inputs and outputs. It can make assessments and carry out tasks based on the information it receives. The data received in the perception step is used in this layer to generate predictions and provide insights [8].
- **Application layer:** The application layer of IoT infrastructure is responsible for data analysis to solve business issues or achieve specific goals. The application layer provides customized functionality to meet the unique requirements of end users. The applications and services in this layer are constructed on top of the processing layer. Software tools facilitate converting data from the processing layer into meaningful information that humans can understand or use by automated processes [9].
- **Business layer:** The business layer acts as the central point where choices and solutions are developed from the data analysis conducted in the application layer. The application layer may consist of several instances inside this layer. At the business layer, identifiable patterns from the application layer are used to gain a deeper understanding of business insights, predict future trends, and make operational decisions that improve productivity, security, cost-effectiveness, customer satisfaction, and other important business factors. In addition, the business layer is accountable for overseeing commercial transactions and models related to interconnected devices. It includes the administration of business processes, data analysis, and rules implementation. It serves as the basis for controlling business logic and setting up procedures to achieve all the business goals of an IoT system [5,8,9].
- **Security layer:** The security layer is present across all levels of the IoT architecture and is essential for the efficiency of an IoT solution. Considering that IoT devices often deal with confidential information, it is crucial to implement strong security measures [5,9].
- **Vulnerability:** It denotes the intrinsic weaknesses of a system or its design, which permit unauthorized entities to execute commands, access data without appropriate authorization, and possibly initiate denial-of-service attacks. Such vulnerabilities can be detected across various domains of IoT systems. They may appear in the system's hardware or software, the protocols and procedures implemented within these systems, and even in the behaviours and actions of the users interacting with the system [4].
- **Exposure:** It refers to a problem or mistake within the system configuration that allows an unauthorized person to undertake actions to obtain information [4].
- **Threats:** It refers to an intentional or unintentional action that exploits vulnerabilities within a system [4].
- **Attacks:** It is defined as deliberate actions to damage a system or disrupt its normal operations by exploiting vulnerabilities using various strategies and tools. Attackers engage in these hostile acts to achieve specific goals, which may be motivated by personal gratification or financial gain [4].

2.2. Common IoT vulnerabilities

There are several vulnerabilities in IoT networks, but according to OWASP, ten significant vulnerabilities make IoT devices susceptible to attacks are (Table 1) [10]:

Table 1
Top 10 OWASP IoT vulnerabilities.

Rank	Vulnerability	Description
1	Weak, guessable, or hardcoded passwords	Inadequate credentials susceptible to brute force attacks or publicly accessible, including backdoors in firmware or client software.
2	Insecure network services	Presence of unnecessary or vulnerable network services, particularly online access, threaten information confidentiality, integrity, and availability.
3	Insecure ecosystem interfaces	Presence of insecure online, backend API, cloud, or mobile interfaces outside the device ecosystem, potentially compromising device security.
4	Lack of secure update mechanism	Inability of devices to undergo secure updates, lacking firmware validation, secure delivery, anti-rollback procedures, or alerts on security changes.
5	Use of insecure or outdated components	Utilization of outdated or vulnerable software or hardware components, potentially exposing devices to unauthorized access.
6	Insufficient privacy protection	Insecure storage or access of user's personal information within the ecosystem.
7	Insecure data transfer and storage	Absence of encryption or access control measures for sensitive data throughout the ecosystem.
8	Lack of device management	Absence of adequate security support for production devices, leading to deficiencies in asset and update management.
9	Insecure default settings	Distribution of devices with unsecured default configurations or limited user control over settings.
10	Lack of physical hardening	Absence of physical security measures, enabling attackers to access critical information or assume local control.

- **Weak/default passwords:** Absence of a strong password recovery system; weak or default password; non-implementation of stricter password rules; inability to change the username and password associated with the account.
- **Insecure network services:** Adversaries exploit vulnerabilities in IoT devices' communication protocols and services to gain unauthorized access and undermine the confidentiality of sensitive information transmitted between the device and a server.
- **Insecure ecosystem interfaces:** The vulnerability of the device or its connected components arising from the insecure web, backend API, cloud, or mobile interfaces in the external ecosystem.
- **Lack of secure update mechanism:** Insufficient ability to update devices securely. These deficiencies encompass the absence of device-based firmware validation, the unsecured transmission of data without encryption, the lack of anti-rollback methods, and the failure to provide notifications regarding security changes resulting from updates.
- **Use of insecure or outdated components:** The utilization of obsolete or insecure software components or libraries that could expose the device to potential attacks. This entails the utilization of external software or hardware obtained from a compromised supply chain, as well as the unsecured modification of system platforms. The security of the IoT ecosystem may be compromised by vulnerabilities in software dependencies or outdated systems.
- **Insufficient privacy protection:** Users' personal information is saved on the device or in the ecosystem and used accidentally, improperly, or illegally. Information about one's health, energy use, and driving habits can fall into this category of privacy concerns. Privacy is at risk without adequate safeguards, and there can be legal consequences for failing to take the necessary precautions.
- **Insecure data transfer and storage:** The absence of encryption or access control for sensitive data at any point in the ecosystem, including whether it is stored, transferred, or processed. Data is critical in ensuring the reliability and integrity of IoT applications since it is used in automated controls and decision-making processes. Unauthorized access or usage will result in adverse consequences.
- **Lack of device management:** Lack of security support for production-ready devices, including asset management, update management, secure decommissioning, systems monitoring, and response capabilities. Unauthorized devices can access business networks, monitor activity, and intercept data if exposed to the IoT ecosystem.
- **Insecure default settings:** Systems or devices that lack the ability to enhance system security by restricting users from modifying configurations or that come with insecure default settings. Once the settings have been acquired, attackers can exploit hardcoded default passwords, concealed backdoors, and weaknesses in the device firmware. The user encounters difficulty in simultaneously modifying various parameters.
- **Lack of physical hardening:** The absence of physical safeguards allows potential attackers to get sensitive information that could be utilized in subsequent large-scale attacks or local device takeover. IoT devices are deployed in distant and dispersed environments. An attacker can disrupt the services offered by IoT devices by gaining access to the physical layer and making alterations.

2.3. IoT or IT attacks

Unlike conventional information technology (IT) attacks, IoT attacks provide distinct issues that necessitate specific security solutions to minimize the associated dangers fully. The distinction between IT and IoT attacks is outlined in Table 2. Some of the different ways are:

- **Attack surface:** IoT devices often possess limited processing capabilities and resources, resulting in potential deficiencies in security features compared to traditional IT systems. Consequently, IoT devices are more susceptible to attacks due to reduced defences [4].
- **Diversity of devices:** The wide array of IoT device types, varying in form factor, operating systems, and network connectivity, complicates establishing standardized security measures. This diversity renders certain devices more prone to vulnerabilities and targeted attacks [4].
- **Physical impact:** Many IoT devices are integral to critical infrastructure or life-sustaining systems, such as medical equipment, thus exposing them to cyberattacks with severe physical ramifications. In contrast, typical IT attacks aim to compromise data integrity or disrupt services [4].
- **Legacy devices:** IoT devices often have extended lifespans, resulting in a proliferation of older, unsupported devices. The inability of legacy devices to receive software updates or security patches renders them particularly vulnerable to exploitation or compromise [4].

3. Related works

Although there has been a lot of research on attacks targeting the IoT, comprehensive literature on classifying these attacks is currently lacking. Therefore, our prior study introduced a new classification system that analyzes existing surveys and taxonomies [4].

Table 2
IoT attacks vs IT attacks.

Category	IoT attacks	IT attacks
Attack surface	Limited resources, higher vulnerability.	Robust security, lower vulnerability.
Diversity of devices	Varied types, complex security.	Standardized, simplified security.
Impact	Harmful physical consequences.	Data theft, service disruption.
Legacy devices	Older devices, no updates, higher risk.	Regular updates, lower risk.

This section first provides an overview of previous research on IoT attacks and then discusses the methods utilized to carry out IoT attack detections, including machine and deep learning.

Many academic publications thoroughly analyze several aspects of risks and attacks in the IoT field. The mentioned literary works [11–15] extensively examine the classification of risks and intrusions linked to the IoT. These papers primarily focus on two main categories: the architectural features of the IoT and the protocols and standards used in the IoT area. Although there is a wealth of literature on threats and attack taxonomies, it is essential to highlight that just a few studies specifically focus on viable solutions.

However, it is essential to mention that the works discussed above do not offer any solutions to the risks and threats that emerge from the extensive use of pervasive technologies like blockchain (BC), fog computing (FC), edge computing (EC), and machine learning (ML). The authors have conducted surveys on diverse pervasive technologies for analyzing risks and attacks in a fragmented manner, as documented in the following sources: [11,16–19], and [20].

The authors of the study [21] did thorough research on security vulnerabilities in the IoT and presented an overview of machine learning methods used to counter these attacks. The authors analyzed 78 publications published until 2017, focusing on the solutions, issues, and areas of research that have not yet been addressed in this field. The authors conducted a survey [22] in 2018 to explore several attack strategies that target the IoT. These models include spoofing attacks, denial-of-service attacks, jamming, and eavesdropping. The authors also suggested possible security techniques to reduce these risks, such as IoT authentication, access control, malware detection, and safe offloading. The security solutions proposed in this study prominently included utilizing machine learning techniques [23].

3.1. Different techniques used to perform IoT attack detections

Identifying and mitigating these attacks is crucial to protecting IoT ecosystems’ security. Several methods have been developed to address the problems of recognizing and responding to attacks. The following methods apply to the identification of attacks in the IoT domain. Also, Fig. 2 provides a thorough overview of several techniques for detecting attacks in the IoT domain.

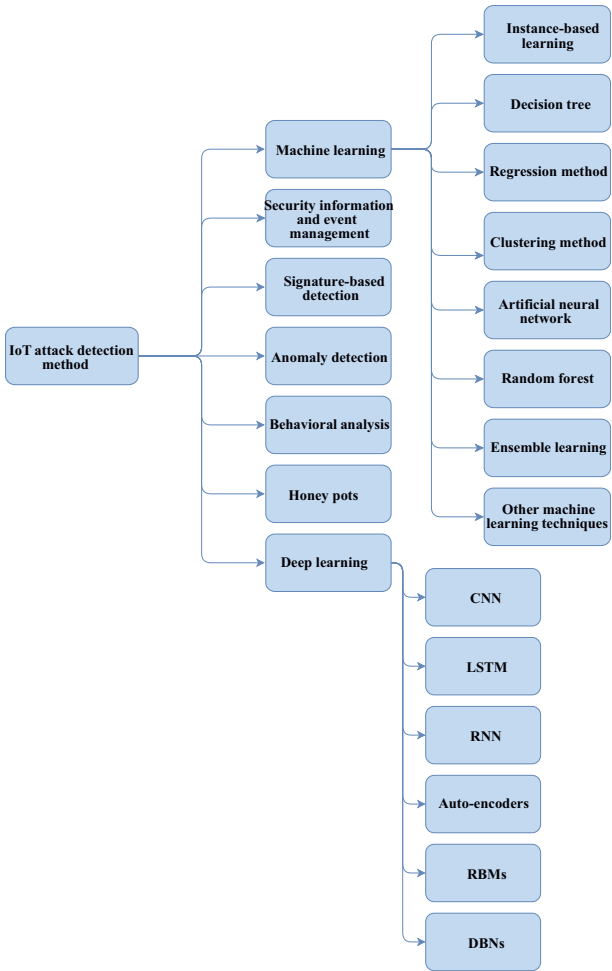


Fig. 2. Overview of available IoT attack detection and identification methods.

- **Anomaly detection:** Anomaly identification, also known as outlier or event detection, is the analytical process of identifying unusual situations inside a particular system. The anomaly detection algorithms assess incoming traffic at multiple levels, ranging from the IoT network level to the data center. Anomaly detection is crucial because it enables the identification and analysis of anomalies within IoT data. Although rare, these anomalies can offer useful insights and practical information in many sectors, including healthcare, industry, finance, transportation, and energy. Anomaly detection in the IoT is employed in the betting and gambling sector to discover insider trading cases by analyzing trade activity patterns [24].
- **Behavioural analysis:** Dynamic code analysis refers to identifying and resolving issues with potentially harmful software within a physical or virtual setting. The program's source code is run with different test inputs to identify security vulnerabilities that may occur due to its code when interacting with other programs or systems. Dynamic analysis is a technique used to study the behaviours of attacks on IoT devices [25]. Utilizing behaviour analysis for detecting IoT attacks offers numerous benefits compared to static analysis. Dynamic analysis can identify known and zero-day threats by examining similar patterns of behaviour exhibited by several attackers. Dynamic analysis is performed by employing sandbox tools like Cuckoo Sandbox or CWSandbox, which allow for the monitoring of malware behaviours in real-time [25].
- **Signature-based detection:** Signature-based detection (SGD) requires security professionals to create predefined rules or signatures to recognize known attack patterns. This method is especially efficient in identifying well-known attacks with signatures stored in the database. On the other hand, the database cannot identify unknown attacks without signatures.
- **Honey pots:** A honeypot is a cybersecurity tool that creates a realistic and valuable network to lure potential attackers. It functions within a network environment that is both isolated and segregated. The aforementioned system can be seen as a simulated entity created to imitate a genuine system to attract potential attackers to interact with it. This method allows for the surveillance of the subsequent interaction between the attackers and the compromised device [26].
- **Security information and event management:** Security information and event management (SIEM) is a security system that assists enterprises in detecting and addressing any security threats and weaknesses, preventing potential disruptions to business operations. Security information and event management (SIEM) systems are essential tools for corporate security teams to detect anomalies in user behaviour. Moreover, these systems utilize artificial intelligence (AI) to simplify and automate specific labour-intensive operations associated with detecting possible threats and subsequent incident response [27].
- **Machine learning:** Machine learning (ML) approaches are crucial in identifying and reducing IoT attacks. These techniques use different algorithms to recognize unusual patterns in network traffic and device activity. The algorithms are trained using extensive datasets, including regular and malicious IoT traffic. This allows them to learn about unique characteristics that distinguish various attacks. Machine learning models, such as decision trees, support vector machines, and random forests, can accurately categorize network traffic as benign or malicious by analyzing patterns they have learned. Furthermore, machine learning algorithms can adjust to changing attack techniques by consistently retraining on updated datasets, thus improving their ability to identify attacks as time goes on. Furthermore, intrusion detection systems that utilize machine learning may function in real-time, promptly notifying security administrators when they identify suspicious behaviours. This allows for quick reactions to possible attacks [4].
- **Deep learning:** Deep learning (DL), a branch of machine learning (ML), provides sophisticated capabilities for identifying attacks in the IoT by automatically extracting complex features from raw data without requiring manual feature engineering. CNNs and recurrent neural networks (RNNs) are frequently used in deep learning-based intrusion detection systems to secure the IoT. CNNs are highly effective at identifying spatial patterns in network traffic data, but RNNs are skilled at capturing temporal relationships in sequences of device action. Using deep learning models, IoT security systems can attain enhanced precision in detecting and defending against advanced attack strategies. Deep learning algorithms can acquire hierarchical data representations, enabling them to identify intricate and nuanced attack patterns. Furthermore, deep learning models can process and analyze large-scale datasets efficiently. They can adjust and perform effectively in dynamic IoT contexts. This makes them highly suitable for identifying familiar and new IoT attacks [4].

To summarize, although there is a significant amount of study on IoT threats, there is a lack of comprehensive literature specifically focusing on their classification. In our earlier study, we presented a new classification method that combines existing surveys and taxonomies to fill this gap. Although many academic articles analyze the various aspects of risks and threats in IoT, there is a lack of focus on practical solutions. Furthermore, the widespread adoption of advanced technologies such as blockchain, fog computing, edge computing, and machine learning raises additional security issues that require further investigation. However, the initiatives mentioned in our assessment offer useful insights into possible remedies against attacks on the IoT.

4. Proposed model

This section presents the motivation for designing a model architecture called self-attention-based 1D-CNN-LSTM, using a CNN combined with an LSTM model enhanced with a self-attention mechanism to identify IoT attacks in TCP network traffic flow data. A one-dimensional convolutional neural network (1D CNN), compared with 2 D CNN, is a specific neural network that employs convolutional layers operating in one dimension. It analyzes data that follows a temporal or sequential pattern, where each data point consists of only one set of features.

The suggested architecture enables efficient detection of IoT attacks without requiring feature engineering. The primary objective of our initial discussion is to examine the rationale behind the proposed strategy. In this context, we emphasize the fundamental significance of our proposed approach. In the next section, we present a comprehensive outline of the different components of our suggested approach.

The proposed approach consists of two distinct steps. The initial phase involves converting data, extracting features, generating datasets for training and testing the model, and developing the model itself. The second module involves training the model using the training dataset, incorporating all features, and determining feature importance by utilizing Shapley additive explanations (SHAP) values. Subsequently, the model is retrained using a decreased set of features while maintaining comparable performance metrics to the original model.

4.1. Pre-processing-stage I

This section will provide a detailed analysis of the components involved in the pre-processing stage technique, and Fig. 3 provides a high-level overview of the pre-processing workflow.

- **Data collection:** In data acquisition, we necessitated benign and malicious pcap files sourced from an IoT attack dataset, significantly facilitating the advancement of security analytics applications within IoT environments. Specifically, we employed the CIC IoT dataset 2023 for training, validation, and testing. Additionally, after completing the training phase, we utilized eight additional IoT attack datasets to evaluate the efficacy of the developed model [28].
- **Data conversation and feature extraction:** As previously outlined in Section 4.1.1, we employed CIC flower, a network traffic flow generator and analyzer, to extract temporal statistical features from raw pcap files. Subsequently, we extracted over 83 features from these files and stored them in CSV format, facilitating their utilization in both model training and testing phases.
- **Data filtration, imputation and creation:** We implemented a filtering mechanism on the dataset, specifically retaining rows associated with an assigned protocol number of 6, indicative of the TCP protocol [29]. In order to handle the missing data in the “Flow Bytes/s” and “Flow Packets/s” columns of our dataset, we choose to utilize KNN Imputation to fill in these missing values. After combining these CSV files, we created the required dataset.
- **Model creation:** Described in detail in Section 4.1.3.

4.1.1. Feature extraction using CICFlowMeter

CICFlowMeter is a tool that generates and analyzes network traffic flow. This tool enables the creation of bidirectional flows, where the first packet determines the direction of data transmission. As a result, over 83 statistical network traffic features, such as duration, number of packets, number of bytes, and length of packets, can be calculated independently for both the forward (source to destination) and backward (destination to source) directions.

Other capabilities encompass choosing features from the available feature list, incorporating new features, and managing the duration of flow timeout. The application generates a CSV file with six columns: FlowID, SourceIP, DestinationIP, SourcePort, DestinationPort, and Protocol. The file contains over 83 network traffic analysis elements. It is essential to understand that TCP flows typically end when the connection is torn down using a FIN message, but a flow timeout ends UDP flows. The individual scheme can provide an arbitrary value for the flow timeout, such as 600 s for both TCP and UDP [30].

4.1.2. Data pre-processing

During the pre-processing stage, we identify a dataset that consists of raw benign and malicious pcap files in order to extract features. The CICflowMeter can be executed either through the command line or within Java integrated development environments (IDEs) such as IntelliJ or Eclipse. When we start the application, we have the option to select either offline or online mode. The offline option allows us to input raw Pcap data into the application for analysis. During this process, we extract features from the Pcap files and save them as CSV files. From the Pcap files, we can extract 83 temporal statistical features, which is a significantly larger number compared to the features provided in the pre-processed CSV files by the dataset authors.

In addition, we have applied a filter to the dataset so that it would only include rows where the assigned protocol number is 6, which corresponds to the TCP protocol [29]. Due to missing values in our dataset, we employed KNN imputation to fill in these missing values.

K-nearest neighbours (KNN) is a machine learning technique for classification and regression tasks. Additionally, it can be utilized to impute missing data. The *K*-nearest neighbours (KNN) imputation approach involves identifying the *K*-nearest neighbours to the observation with missing data and subsequently imputing the missing values based on the non-missing values in those neighbours [31].

K-nearest neighbours (KNN) imputation is a highly favoured technique for replacing missing data in time series because it offers numerous significant benefits. Firstly, its non-parametric character enables it to accommodate a wide range of data distributions observed in time series without making assumptions about underlying data patterns. KNN imputation utilizes the principle that data points with similar characteristics typically have similar values. This method estimates missing values by considering the values of

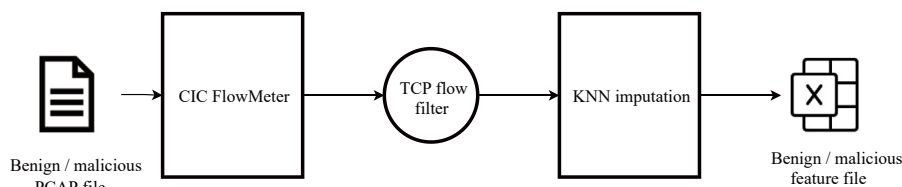


Fig. 3. Data pre-processing.

nearby points, thus maintaining the local structure of the data. Unlike approaches that presume linearity, KNN imputation is appropriate for time series data with nonlinear or complex connections between variables.

Furthermore, its ability to withstand outliers guarantees stability in irregular data points, as it prioritizes nearby clusters rather than overall patterns. KNN imputation can adapt to changing patterns in time series data by considering the nearest neighbours inside a sliding window. This makes it highly skilled at capturing evolving data dynamics. Furthermore, the straightforward application and little need for adjusting parameters, such as determining the number of neighbours (k) and distance metric, enhance its attractiveness as a powerful and adaptable method for filling in missing values in time series datasets [32].

4.1.3. Model structure

In this section, we examine the components utilized in the model's structure. This model incorporates convolutional layers, residual blocks, LSTM layers with an attention mechanism, and some final processing and output layers. Let's break down each module's purpose and workings:

- **Input layer:** This layer defines the input shape required to process the data for model training and testing. For our model, we have identified the input shape for the input layer to be $(feature_len, 1)$, where $feature_len$ is the number of columns in one batch.
- **Residual blocks:** Start the model with CNN based residual block which is used to deepen the network without losing the ability to train effectively.
- **Max pooling:** Reduces the spatial dimensions of the output from convolutional layers, summarizing features.
- **Dropout:** Applied after pooling and LSTM layers to prevent overfitting by randomly dropping units during training.
- **LSTM layers:** Processes the output of convolutional layers to learn from the temporal patterns in the data. $return_sequences = True$ keeps the time dimension for attention processing.
- **Attention layers:** Applied to the output of the LSTM layers to focus the model's learning on important temporal elements.
- **Flatten:** Converts the multi-dimensional output of the attention layer into a single-dimensional array suitable for input to the fully connected layer.
- **Dense layer:** A dense layer serves as the intermediate stage where the spatially distributed features extracted by convolutional, pooling layers and LSTMs are flattened into a single vector representation. By doing so, dense layers enable the network to learn high-level abstractions and relationships among the extracted features, making them more suitable for classification or regression tasks. These intermediate dense layers typically incorporate non-linear activation functions like ReLU to introduce non-linearity and capture complex patterns in the data. Additionally, they may include dropout or batch normalization layers to regularize the network and prevent overfitting.
- **Output layer:** A dense layer that outputs the final prediction of the model. The activation function is chosen based on the task. Since we are performing binary classification, we are using the sigmoid function.

Figs. 4 and 5 provide a high-level overview of the model architecture. This architecture is particularly effective for complex sequence modelling tasks that benefit from spatial feature extraction and the ability to remember and emphasize important parts of the sequence data over time.

4.1.4. Training and tuning-stage II

The second module involves training the model using the training dataset, incorporating all features, and determining feature importance by utilizing SHAP values. Subsequently, the model is retrained using a decreased set of features while maintaining comparable performance metrics to the original model.

- **Hyperparameter tuning:** Hyperparameter tuning, also known as hyperparameter optimization, refers to selecting the optimal hyperparameters for a machine learning model to maximize its performance on a given dataset. Hyperparameters are parameters set

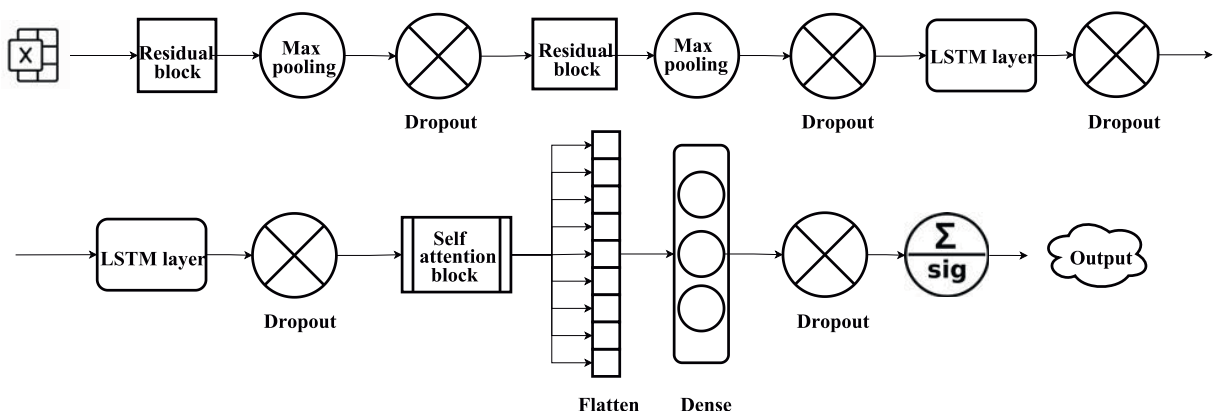


Fig. 4. Model architecture.

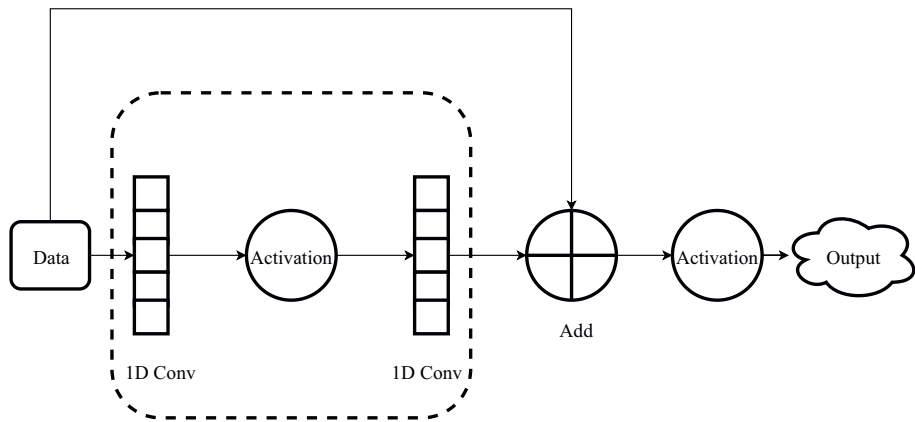


Fig. 5. Residual block.

before the learning process begins and control aspects such as the complexity of the model, the regularization strength, and the learning rate. Unlike model parameters learned during training, hyperparameters are not learned from the data and must be specified by the user [33].

Grid search is a hyperparameter tuning technique to find the optimal combination of hyperparameters for a machine learning model. It involves defining a grid of hyperparameter values and exhaustively searching through all possible combinations of these values to identify the combination that yields the best performance on a chosen evaluation metric.

Using grid search, we have successfully determined five optimal combinations of hyperparameters for training, validation, and testing. Among the five options, we have selected the hyperparameter configuration that performs the best for calculating feature importance, retraining the model, and compressing it. This selection was based on testing it on external datasets. Below is Table 3 of hyperparameters utilized for training the model.

- **Model training and testing:** To facilitate model training, we divided the dataset into two subsets-training and testing-in an 80:20 ratio. This was achieved using the `train_test_split` function from the sklearn library. In addition, we partitioned the data in the training subset into separate training and validation sets using an 80:20 ratio. This division assisted in training the model according to the method described in reference [34].
- **SHAP feature importance:** We employ SHAP feature importance scores to identify the most impactful features in our model. Subsequently, we refine the model by training it with a reduced feature set, guided by insights drawn from the SHAP feature importance graph, particularly focusing on the differences between adjacent bar levels. The SHAP feature importance graph in Fig. 6 provides a visual representation of these insights.

SHAP feature importance provides a comprehensive method for understanding the impact of individual features on a machine learning model's predictions. Unlike traditional methods like permutation importance, which assesses feature importance by measuring the decrease in model performance when a feature's values are randomly permuted, SHAP values consider the interaction between features and provide a more nuanced understanding of their influence on predictions [35,36].

While permutation importance offers a straightforward measure of feature importance, SHAP values capture the contribution of each feature in the context of other features, allowing for a more accurate and interpretable assessment of their impact. In situations where feature interactions are significant in model predictions, SHAP feature importance provides more insightful and reliable results. However, permutation importance may suffice for simpler models with fewer interactions between features and could be computationally less intensive. Ultimately, the choice between SHAP feature importance and permutation importance depends on the complexity of the model and the importance of capturing feature interactions for accurate interpretation [36].

While SHAP values typically provide local explanations for individual predictions, aggregating these values across a dataset offers insights into the global importance of each feature. Global feature importance quantifies the overall impact of features across all predictions, measuring how significant each feature is in the model's decision-making process.

- **Model retraining and model compression:** After identifying the reduced set of features, we build and train the model on it. Next, we compare the performance characteristics of the initial and reduced set models. If the performance of the reduced model is comparable to the initial model, we will use the reduced model for deployment.

Table 3
Hyperparameters list.

No	Activations	Losses	Optimizers	Batches	Epochs	Shuffles
1	ReLU	Binary cross-entropy	Adam	32	20	True
2	ReLU	Binary cross-entropy	Adam	16	20	True
3	Leaky ReLU	Binary cross-entropy	Adam	16	20	True
4	ReLU	Binary cross-entropy	RMSProp	16	20	True
5	ReLU	Binary cross-entropy	RMSProp	8	20	True

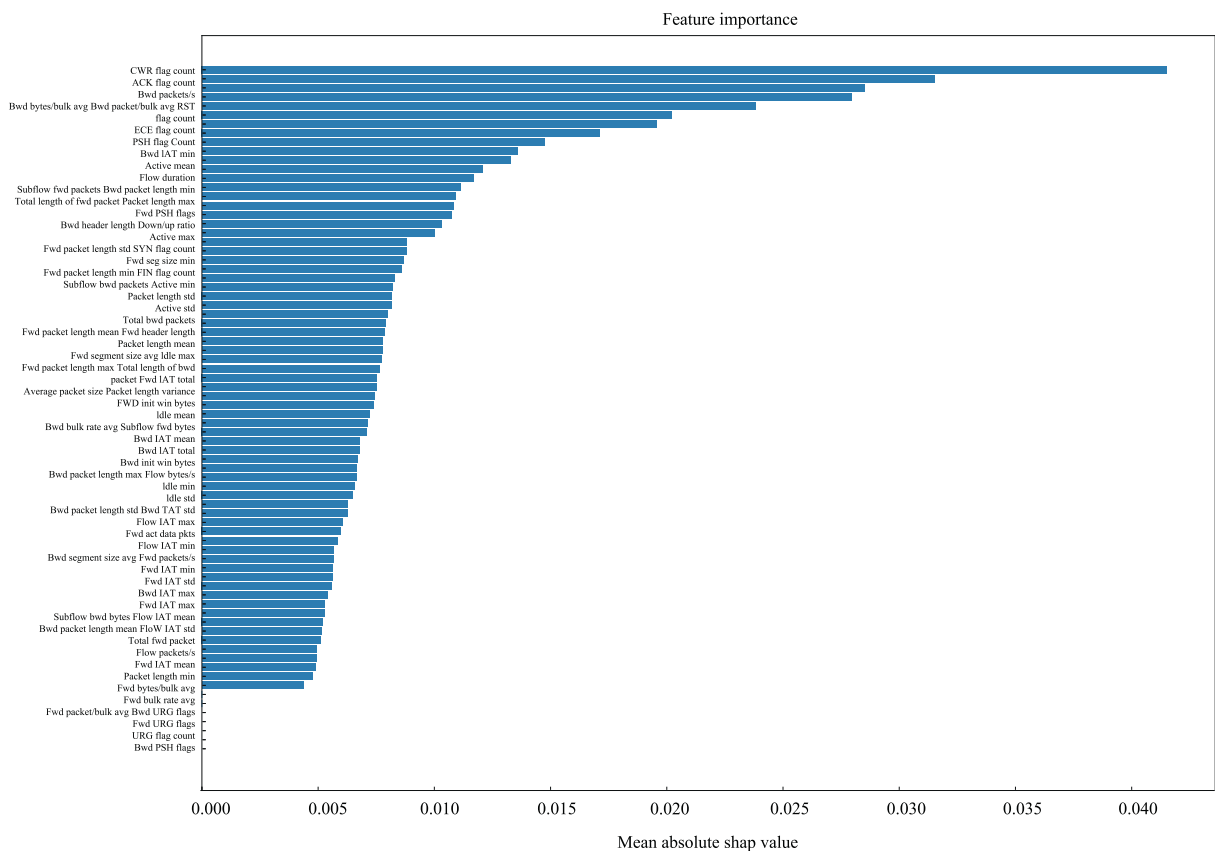


Fig. 6. Feature importance graph.

5. Experiments & results

This section provides a detailed overview of the experimental framework and the dataset utilized for conducting the experiments and obtaining the results. Subsequently, an exhaustive examination of the features used in the proposed method is performed, with a comprehensive list of all the features included. As outlined in Section 4, this section also demonstrates the experimentation and results of the proposed self-attention-based 1D-CNN-LSTM deep learning technique.

Further in the section, the implementation results of the proposed model are presented in detail, along with a thorough discussion of the performance metrics on externally augmented datasets. Finally, the outcomes of these experiments validate the effectiveness of the proposed system.

5.1. Experimental setup

The implementation and execution of all the modules were processed on Windows 11, 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30 GHz 2.30 GHz, 64 GB RAM with a graphic card of NVIDIA Geforce RTX 3060 4 GB. All the data preprocessing and feature extraction were implemented using Python scripts and public libraries.

5.2. Datasets

Nine augmented datasets have been generated using the publicly available IoT datasets to train, test, and evaluate the proposed model to maximize the variety of attacks and benign activities. Below is a summary of datasets augmented during preprocessing for training, validation, and model testing, summarized in table Table 4.

- **ACI IoT network traffic dataset 2023:** This study aims to introduce a novel dataset tailored explicitly for machine learning (ML) applications in the domain of IoT network security. The research provides a distinctive and realistic dataset for training and assessing ML models designed for IoT network environments. By addressing an existing gap in available resources, this dataset aims to drive progress in ML-based solutions, thereby enhancing the security of IoT operations.

Table 4
Augmented datasets.

Year	New dataset name	Original dataset name	Malicious activities	Real/simulated	Dataset size
2023	CIC-BCCC-NRC_IoT-2023	CIC IoT 2023	Yes	Real	108.58 GB
2023	CIC-BCCC-NRC_ACI-IoT-2023	ACI IoT network traffic dataset 2023	Yes	Real	338.9 MB
2024	CIC-BCCC-NRC_IoMT-2024	CIC IoMT 2024	Yes	Real	19.25 GB
2022	CIC-BCCC-NRC_Edge-IIoTset-2022	Edge-IIoTset	Yes	Real	1.55 GB
2022	CIC-BCCC-NRC_IoT-2022	CIC IoT 2022	Yes	Real	60.6 MB
2019	CIC-BCCC-NRC_IoT-HCRL-2019	IoT network intrusion dataset	Yes	Real	46.3 MB
2020	CIC-BCCC-NRC_MQTTIoT-IDS-2020	MQTT-IoT-IDS-2020	Yes	Simulated	1.45 GB
2021	CIC-BCCC-NRC_TONIoT-2021	TON IoT 2021	Yes	Real	696.2 MB
2022	CIC-BCCC-NRC_UQ-IoT-2022	UQ IoT 2022	Yes	Real	2.31 GB

The experimentation phase was conducted within the Internet of Things Research Lab (IoTRL) at the Army Cyber Institute (ACI), meticulously crafted to replicate a realistic home environment. The IoTRL mirrors the conditions of a typical home IoT network, encompassing a diverse range of both wired and wirelessly connected devices, including various IoT devices and conventional networked devices commonly found in home settings. Home Assistant, free and open-source software for home automation, was seamlessly integrated within the lab as a central control system to streamline the control and management of IoT devices [37]. Data collection involved strategically deploying network sniffers and monitoring points throughout the lab to capture both wired and wireless network traffic, thus creating a realistic and dynamic environment for experimentation. The dataset stands out in the IoT security research landscape due to several vital contributions: The inclusion of a dynamic mix of IoT devices, simulation of a realistic home environment within the IoTRL, focus on behavioural analysis of IoT devices in normal and adversarial scenarios, and incorporation of multi-modal data representation including network traffic patterns, device communication, and device-specific characteristics. These contributions enable a comprehensive evaluation of IoT security, surpassing the limitations of existing datasets with a single focus [37].

- **CIC IoT dataset 2022:** This study endeavors to produce an advanced dataset to profile, analyze behaviour, and test vulnerabilities of various IoT devices utilizing protocols such as IEEE 802.11, Zigbee-based, and Z-wave. Regarding network configuration, our lab network was set up with a 64-bit Windows machine equipped with two network interface cards—one linked to the network gateway and the other to an unmanaged network switch. Wireshark, an open-source network protocol analyzer, concurrently monitors both interfaces, capturing and saving packet data. This setup allows IoT devices requiring an Ethernet connection to the switch [38]. Additionally, a smart automation hub, Vera Plus, is linked to the unmanaged switch, establishing our wireless IoT environment to cater to devices compatible with Wi-Fi, Zigbee, Z-wave, and Bluetooth. Data collection involved capturing network traffic of IoT devices passing through the gateway using Wireshark and dumpcap across six experiment types: power, idle, interactions, scenarios, active, and attacks. Each experiment type is delineated as follows: power involved individually powering on devices in isolation and capturing network traffic; idle captured network traffic during overnight periods of minimal lab activity; interactions extracted all possible functionality from IoT devices, capturing corresponding network activity; scenarios simulated network activity within a smart home environment with various device combinations; active captured network communication throughout the day, allowing fellow researchers to interact with devices; attacks entailed performing flood and RTSP-brute force attacks on select devices and capturing their attack network traffic [38].
- **CIC IoMT dataset 2024:** The primary objective of this study is to propose a realistic benchmark dataset to facilitate the development and evaluation of security solutions for the Internet of Medical Things (IoMT). This involved executing 18 attacks on an IoMT testbed comprising 40 devices (25 real and 15 simulated), considering the diverse protocols prevalent in healthcare settings, such as Wi-Fi, MQTT, and Bluetooth. These attacks were categorized into five classes, DDoS (Distributed DoS), DoS (Denial-of-Service), Recon, MQTT, and spoofing, to establish a foundational benchmark that complements existing contributions and aids researchers in exploring novel security mechanisms, including machine learning (ML). Data collection involved the utilization of a network tap to capture traffic between the switch and the Wi-Fi/MQTT-enabled IoMT devices, with real-time packet duplication facilitated by the device to construct the security and profiling dataset. Furthermore, a malicious PC and a smartphone were employed to capture malicious and benign data for Bluetooth low energy (BLE) enabled devices [39].
- **Edge-IIoTset:** It is a comprehensive and realistic cyber security dataset tailored for IoT and IIoT applications, intended for utilization in machine learning-based intrusion detection systems operating in centralized and federated learning modes. The dataset is structured across seven layers: cloud computing layer, network functions virtualization layer, blockchain network layer, fog computing layer, software-defined networking layer, edge computing layer, and IoT and IIoT perception layer. Each layer incorporates cutting-edge technologies meeting the essential requirements of IoT and IIoT applications, including the ThingsBoard IoT platform, OPNFV platform, Hyperledger Sawtooth, Digital twin, ONOS SDN controller, Mosquitto MQTT brokers, Modbus TCP/IP, among others [40].

Data generation involves diverse IoT devices spanning over ten types: low-cost digital sensors for temperature and humidity, ultrasonic sensors, water level detection sensors, pH sensor meters, soil moisture sensors, heart rate sensors, and flame sensors. Furthermore, the project identifies and analyzes fourteen attacks associated with IoT and IIoT connectivity protocols, categorized into five threat classes: DoS/DDoS attacks, information gathering, man-in-the-middle attacks, injection attacks, and malware attacks. Following the processing and analysis of the proposed cyber security dataset, primary exploratory data analysis is conducted, and the performance of machine learning approaches is evaluated in centralized and federated learning modes [40].

- **IoT network intrusion dataset:** This dataset features various network attacks within an IoT environment, employing two representative smart home devices: the SKT NUGU (NU 100) and the EZVIZ Wi-Fi Camera (C2C Mini O Plus 1080P). All devices, alongside selected laptops or smartphones, were interconnected within a unified wireless network. The dataset encompasses 42 raw network packet files (pcap) captured at distinct intervals. These packet files were obtained using the monitor mode of a wireless network adapter, with subsequent removal of wireless headers facilitated by Aircrack-ng. The dataset includes attacks simulated using tools such as Nmap, excluding those categorized under the Mirai Botnet, for which attack packets were generated on a laptop and subsequently altered to appear as originating from an IoT device [41].
- **MQTT-IoT-IDS 2020:** This study aims to develop a dataset pertaining to the IoT domain, specifically focusing on the MQTT communication protocol, to provide the research and industrial communities with an initial dataset for their applications. The dataset comprises IoT sensors operating on MQTT, each representing various aspects of a real network. Specifically, the MQTT broker is instantiated using Eclipse Mosquitto, and the network consists of 8 sensors. The scenario emulates a smart home environment where sensors collect data on temperature, light, humidity, CO-Gas, motion, smoke, door status, and fan operation at different time intervals, reflecting the diverse behaviour of each sensor. These 8 MQTT sensors are characterized by distinct features, detailed in a table format, with each sensor associated with a data profile and a topic linked to the MQTT broker [42]. Additionally, the sensors are conceptually divided into two rooms, resembling distribution within a smart house, while the MQTT broker is assigned the IP address 10.16.100.73 and port 1883 for clear text communication. The time intervals for data transmission can either be periodic or random, with periodic intervals reflecting cyclic data transmission, such as that of a temperature sensor, and random intervals representing sporadic data transmission, as in the case of a motion sensor triggered by user activity. This consideration enhances the dataset's fidelity, simulating and implementing realistic behaviours characteristic of home automation systems [42].
- **TON IoT:** The TON IoT datasets represent a new iteration of Industry 4.0/IoT and Industrial IoT (IIoT) datasets designed to assess the efficacy and authenticity of diverse cybersecurity applications leveraging artificial intelligence (AI), including machine and deep learning algorithms. These datasets amalgamate heterogeneous data sources derived from telemetry datasets of IoT and IIoT sensors, operating systems datasets encompassing Windows 7 and 10, Ubuntu 14 and 18 TLS, and network traffic datasets. They were compiled from a realistic and expansive network environment established at the cyber range and IoT labs within the School of Engineering and Information Technology (SEIT) at UNSW Canberra @ the Australian Defence Force Academy (ADFA). A novel testbed network was devised specifically for the Industry 4.0 context, encompassing IoT and IIoT networks. It was implemented through multiple virtual machines and hosts operating on Windows, Linux, and Kali systems, facilitating the integration across IoT, cloud, and edge/fog layers. The datasets encapsulate a range of attacking methodologies, including DoS, DDoS, and ransomware, targeting web applications, IoT gateways, and computer systems within the IoT/IIoT network. The datasets were compiled through parallel processing techniques to capture normal and cyber-attack events from network traffic, Windows audit traces, Linux audit traces, and telemetry data originating from IoT services.
- **UQ IoT IDS 2021:** The UQ-IoT-IDS-2021 dataset, provided by the University of Queensland (UQ), Australia, primarily focuses on cybersecurity within the IoT domain. It encompasses benign traffic generated by various IoT devices, including smartphones, smart TVs, IP cameras, smart speakers, and Raspberry Pis, among others, alongside malicious traffic originating from 9 distinct cyberattack types, such as Host Discovery, Port Scanning, Service Detection, ARP Spoofing, Telnet Brute-force, SYN Flooding, UDP Flooding, HTTP Flooding, and ACK Flooding. The raw data was captured utilizing Wireshark within a realistic network environment established at the School of Information Technology and Electrical Engineering, UQ. Subsequently, this raw data underwent processing by the feature extractor in Kitsune to extract 100 temporal-statistical features, enhancing the dataset's utility for cybersecurity research and analysis [43].

5.3. Summary of IoT attacks in training dataset

The selected training dataset includes 14 common IoT attacks as follows, and [Appendix A](#) lists the malicious activities in all selected datasets.

1. **ACK fragmentation:** Fragmented ACK attacks, a variant of ACK & PSH-ACK Flood, use 1500-byte packets to dominate network bandwidth at a low packet rate. These packets pass through routers, ACLs, and firewalls undetected, exhausting resources as they require reassembly. Containing irrelevant data, they aim to crash the target, disrupting services similar to other DDoS attacks [44].
2. **ICMP flood:** An ICMP flood is a DoS attack that uses pings to analyze a device's operation. A “ping flood attack” occurs when attackers overload a network router or IP address with ICMP packets, preventing it from handling legitimate traffic. This exhausts the device's resources (memory, processing power, and interface rate), hindering service to actual requests [45].
3. **ICMP fragmentation:** IP/ICMP fragmentation volumetric DoS attacks flood networks by breaking IP datagrams into smaller packets, which are reassembled during communication. This method meets network size limits, governed by the maximum transmission unit (MTU). Packets exceeding this limit must be fragmented, with only one packet containing essential information. Attackers use IP fragmentation to target communication and security systems, often sending forged fragments that cannot be reassembled, depleting memory resources. These attacks use fake UDP or ICMP packets designed to exceed the MTU, quickly exhausting server resources and making it unavailable for legitimate traffic [46].

4. *PSH ACK flood*: ACK or PUSH ACK packets facilitate bidirectional data transfer between host and client until the session ends. ACK flood attacks target vulnerable servers with bogus ACK packets from non-existent sessions. This forces the server to use resources to verify the packets, leading to decreased performance and service availability [47].
5. *DDoS RST FIN flood*: Clients and hosts use RST or FIN packets to end TCP SYN sessions. In an RST or FIN flood, the targeted server receives numerous counterfeit packets unrelated to its connections. The server must use significant resources to match these packets, reducing performance and causing partial unavailability [48].
6. *SYN flood*: SYN floods, or half-open attacks, are DDoS attacks that deplete a server's resources by flooding it with SYN packets, causing slow or no response to legitimate traffic [49].
7. *Synonymous IP flood*: This attack floods the target server with fake TCP SYN packets using the victim's source and destination addresses, forcing the server to use system resources to process each packet [50].
8. *DDoS TCP flood*: TCP SYN flood attacks exploit the TCP three-way handshake by sending numerous SYN requests without completing the final ACK, causing the server to waste resources on partially formed connections [51].
9. *UDP flood*: A UDP flood attack uses the user datagram protocol to launch a volumetric DoS attack by flooding the victim system with UDP packets to random ports. This forces the host to scan for active programs and respond with ICMP packets, exhausting resources and preventing access for authorized users. Attackers may spoof IP addresses to conceal their location [52].
10. *UDP fragmentation*: A variation of UDP flood, this attack uses the largest allowable packets to overload the channel with fewer packets. These counterfeit packet fragments force the server to waste resources trying to reassemble non-existent packets, leading to resource depletion, server crashes, or channel overflow. This attack is hard to filter and poses a high risk of channel overflow [53].
11. *DNS spoofing*: A hacker uses DNS spoofing to trick a machine or network into communicating with a fake website or server, leading to potential identity theft, financial fraud, malware, and other online issues [54].
12. *HTTP flood*: An HTTP flood is a DDoS attack that uses legitimate HTTP GET or POST requests to overwhelm a web server or application. These volumetric attacks often involve a botnet and do not rely on defective packets, spoofing, or reflection. They require less bandwidth to take down a target and must be carefully adapted to the specific site or application, making detection and prevention difficult [55].
13. *Mirai*: The Mirai botnet software turns IoT devices into “bots” for launching powerful volumetric DDoS attacks. It targets devices with open ports or default credentials, infecting them with malware. Infected devices join the Mirai botnet, allowing attackers to issue commands from a central “command & control” server. This enables simultaneous DDoS attacks on websites, networks, and digital infrastructure using the collective power of all Mirai bots [56].
 - *Mirai GRE-ETH flood*: GRE creates IP-based virtual point-to-point connections and can encapsulate many network layer protocols. DDoS scrubbing services use GRE for mitigation. The GRE-ETH flood attacks routers and switches with excessive GRE and ETH frames, draining CPU, memory, and bandwidth. The goal is to interrupt network services, cause DoS or DDoS, and exploit weaknesses in the target device's traffic management [57].
 - *Mirai GRE-IP flood*: A GRE-IP flood is a network attack that overwhelms a network with GRE and IP packets. The attacker sends numerous GRE-encapsulated IP packets to exhaust network resources, causing congestion, slow speeds, or service disruptions. Goals include interrupting network operations, causing DoS, or exploiting infrastructure weaknesses [57].
 - *Mirai UDP plain*: UDP plain attacks, or UDP flood attacks, are network-based DoS attacks that flood a server or network with UDP packets. Since UDP is connectionless and does not require a handshake, attackers can quickly produce and transmit packets. This overwhelms the target, causing network delays, degraded services, or restricted access. The attack exploits UDP's statelessness, potentially causing packet loss or disorder, further disrupting communication [52].
14. *MITM ARP spoofing*: Attackers intercept network traffic via ARP spoofing (ARP poisoning). After accessing the network, they identify the IP addresses of devices like a workstation and a router. The attacker then sends false ARP responses, making the devices update their ARP cache to reroute traffic through the attacker's device. This allows the attacker to eavesdrop on conversations, hijack sessions, tamper with communication, and initiate DDoS attacks [58].

5.4. Features

In this research, we used CICFlowMeter to extract the network-based features from benign and malicious PCAP files for only TCP network flow data, which have been listed in [Appendix B](#).

5.5. Finalizing the proposed model

In this section, the proposed model is elaborated upon in detail. After extracting features using the CICFlowMeter, the newly created dataset was filtered to include only TCP flows and KNN imputation was employed to address missing values. Subsequently, a self-attention-based 1D-CNN-LSTM mechanism was constructed, and hyperparameter tuning was performed using the grid search technique. This process identified the top 5 hyperparameters indicated in [Table 3](#).

We evaluated five models across nine augmented test datasets. From these, the top best-performing model was selected for SHAP feature importance calculations. These scores were then utilized to identify reduced model retraining and compression features.

After training the top model with this reduced feature set, a comparison was conducted against the initial model. As depicted in [Fig. 7](#), the results indicated only a minor performance decrement between the initial model and those with the reduced feature set.

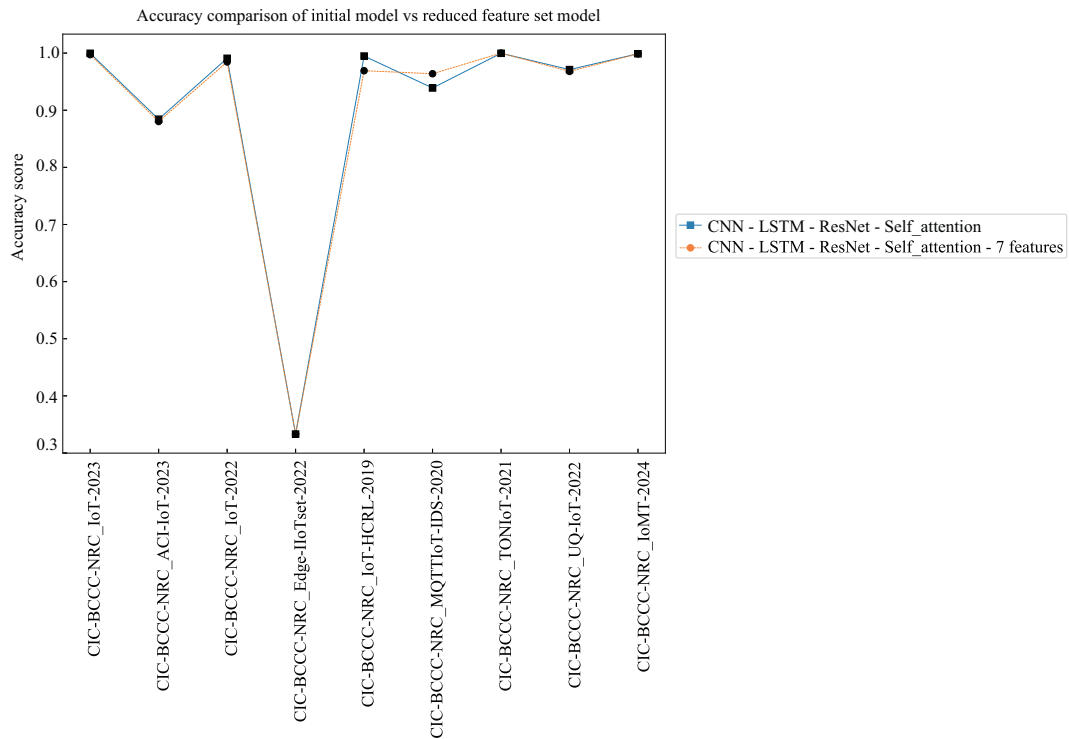


Fig. 7. Accuracy comparison between initial model vs reduced feature set model.

5.6. Experimental results

This section presents our experiment findings regarding identifying malicious activities within network TCP flow data. Due to the predominance of malicious activities over benign ones, a data balancing procedure was necessitated prior to model training. Specifically, we retained all rows corresponding to benign activities and randomly sampled consecutive rows representing malicious activities from each respective group within the dataset. This approach ensured parity in the total count of benign and malicious activities. Consequently, the size of the resulting training set was 356.3 MB.

Drawing upon the insights Table 3 provided, we opted for the most effective hyperparameter configuration to train our model architecture employing 5-fold cross-validation. Following this, we conducted comprehensive tests and comparisons between our model and baseline models across all datasets mentioned in Table 4. The outcomes of these analyses are displayed in Table 5 and Fig. 8.

Table 5

Baseline models evaluation results based on accuracy.

Dataset	CNN	CNN-LSTM	CNN-LSTM-ResNet	CNN-LSTM-ResNet-additive attention	CNN-LSTM-ResNet-self attention
CIC-BCCC-NRC_IoT-2023	0.9999	0.9998	0.9995	0.9998	0.9996
CIC-BCCC-NRC_ACI-IoT-2023	0.8809	0.8850	0.8958	0.8918	0.8842
CIC-BCCC-NRC_IoT-2022	0.9915	0.9912	0.5167	0.6096	0.9908
CIC-BCCC-NRC_edge-IIoTset-2022	0.3273	0.3140	0.3331	0.3331	0.3330
CIC-BCCC-NRC_IoT-HCRL-2019	0.9949	0.9953	0.9294	0.9746	0.9948
CIC-BCCC-NRC_MQTTIoT-IDS-2020	0.9184	0.9587	0.9284	0.9627	0.9639
CIC-BCCC-NRC_TONIoT-2021	0.9992	0.9997	0.9805	0.9934	0.9997
CIC-BCCC-NRC_UQ-IoT-2022	0.9680	0.9709	0.9774	0.9690	0.9711
CIC-BCCC-NRC_IoMT-2024	0.9863	0.9899	0.5045	0.5873	0.9988

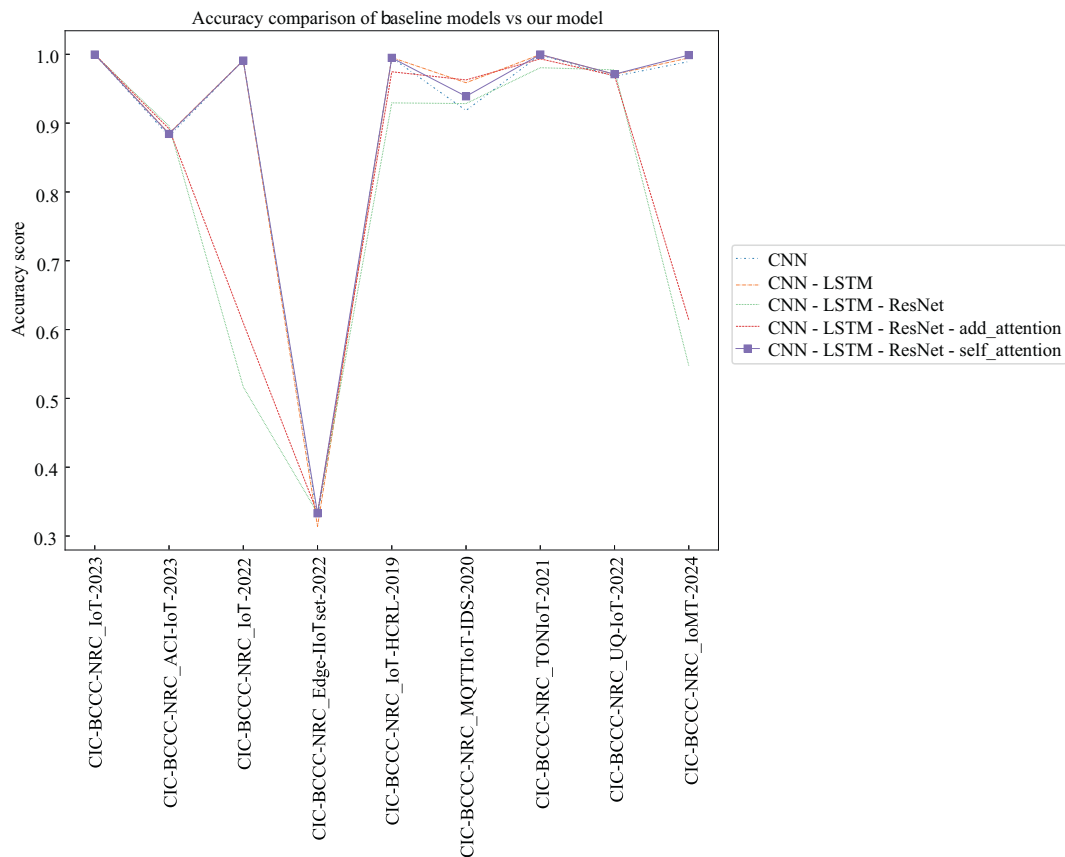


Fig. 8. Accuracy comparison between baseline models and our model.

The training history of our main model and the confusion matrix of the main model are described in Figs. 9 and 10, respectively. After evaluating our model using the CIC-BCCC-NRC_IoT-2023 test data and the dataset mentioned in Table 4, we obtained the results summarized in Table 6. The confusion matrix for the initial model across all datasets is provided in Table 7.

After completing the evaluation process, SHAP analysis is employed to compute feature importance, facilitating the identification of key features that contribute to the model's predictive outcomes. Subsequently, Fig. 6 and Fig. 11 illustrate the summary plot and feature importance scores derived from analyzing a randomly selected subset of 100 test data samples.

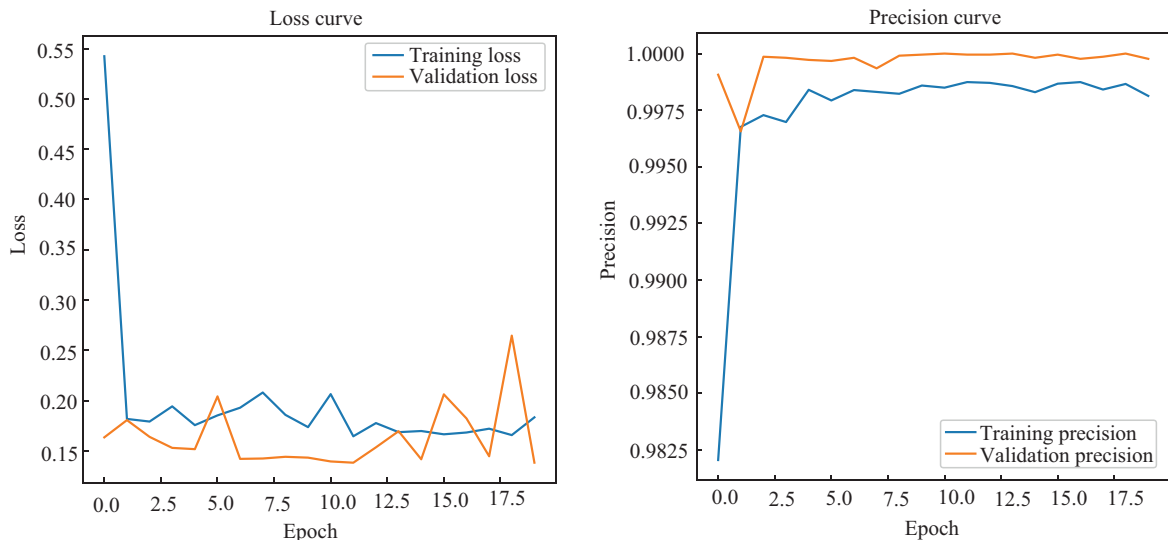


Fig. 9. Our model's training history results.

Table 6
Our model accuracy list.

Dataset	Accuracy
CIC-BCCC-NRC_IoT-2023	0.9996
CIC-BCCC-NRC_ACI-IoT-2023	0.8842
CIC-BCCC-NRC_IoT-2022	0.9908
CIC-BCCC-NRC_Edge-IIoTset-2022	0.3330
CIC-BCCC-NRC_IoT-HCRL-2019	0.9948
CIC-BCCC-NRC_MQTTIoT-IDS-2020	0.9633
CIC-BCCC-NRC_TONIoT-2021	0.9997
CIC-BCCC-NRC_UQ-IoT-2022	0.9711
CIC-BCCC-NRC_IoMT-2024	0.9988

Table 7
Confusion metrics for initial model across all datasets.

Dataset	Accuracy	Precision	Recall	F1 score
CIC-BCCC-NRC_IoT-2023	0.9996	0.9998	0.9995	0.9996
CIC-BCCC-NRC_ACI-IoT-2023	0.8842	0.8848	0.9984	0.9381
CIC-BCCC-NRC_IoT-2022	0.9908	0.9916	0.9993	0.9954
CIC-BCCC-NRC_Edge-IIoTset-2022	0.3330	0.3331	1	0.4997
CIC-BCCC-NRC_IoT-HCRL-2019	0.9948	0.9968	0.9981	0.9974
CIC-BCCC-NRC_MQTTIoT-IDS-2020	0.9633	0.9630	0.9741	0.9685
CIC-BCCC-NRC_TONIoT-2021	0.9997	1	0.9997	0.9999
CIC-BCCC-NRC_UQ-IoT-2022	0.9711	0.9710	1	0.9853
CIC-BCCC-NRC_IoMT-2024	0.9988	0.9989	0.9999	0.9994

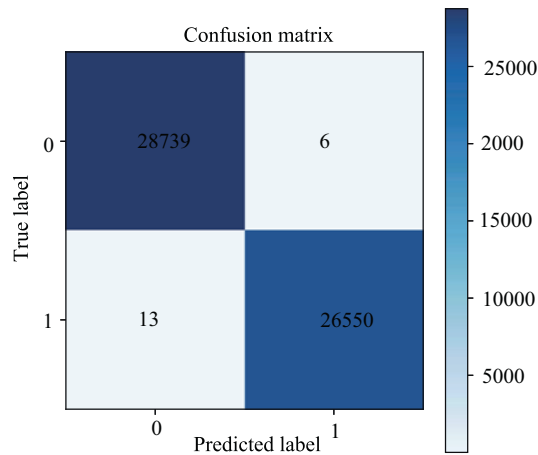


Fig. 10. Our model's confusion matrix results.

Table 8
Reduced set's features accuracy list.

Dataset	4 features	5 features	7 features	10 features	11 features
CIC-BCCC-NRC_IoT-2023	0.9847	0.9963	0.9973	0.9992	0.9995
CIC-BCCC-NRC_ACI-IoT-2023	0.8797	0.8804	0.8803	0.8801	0.8812
CIC-BCCC-NRC_IoT-2022	0.2603	0.2635	0.9845	0.9914	0.9914
CIC-BCCC-NRC_Edge-IIoTset-2022	0.3330	0.3330	0.3330	0.3330	0.3330
CIC-BCCC-NRC_IoT-HCRL-2019	0.9807	0.9027	0.9691	0.8740	0.9715
CIC-BCCC-NRC_MQTTIoT-IDS-2020	0.9639	0.9638	0.9639	0.9642	0.9639
CIC-BCCC-NRC_TONIoT-2021	0.9988	0.9997	0.9999	0.9998	0.9999
CIC-BCCC-NRC_UQ-IoT-2022	0.9680	0.9679	0.9680	0.9642	0.9690
CIC-BCCC-NRC_IoMT-2024	0.2711	0.2747	0.9987	0.9925	0.9905

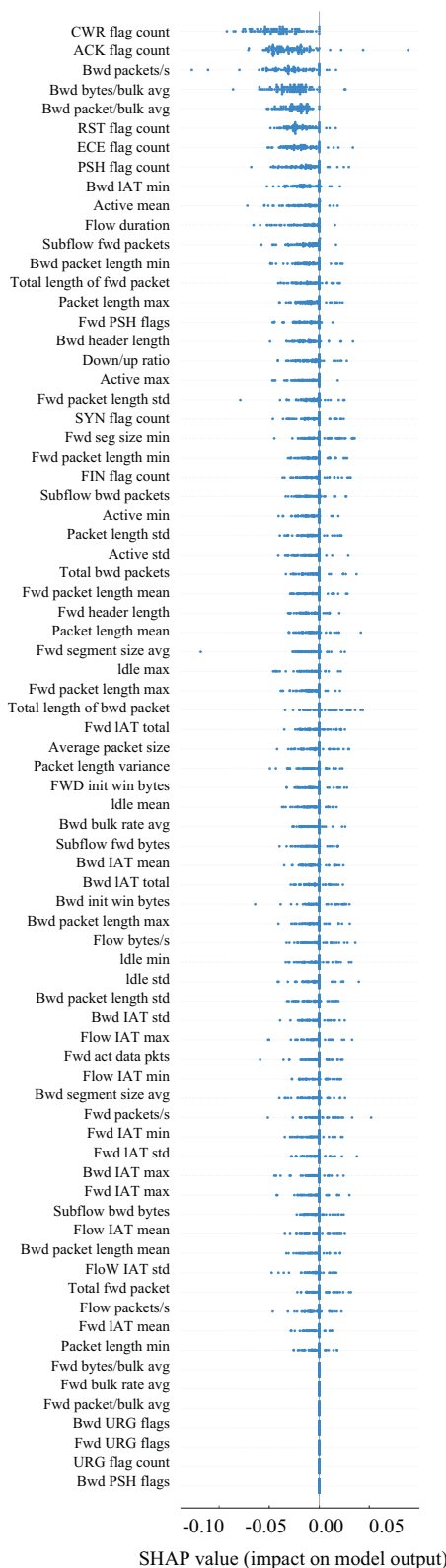


Fig. 11. Shap summary plot for our model (trained on CIC-BCCC-NRC_IoT-2023 dataset).

Due to the standard scaling and balancing of the training dataset, the SHAP values may appear smaller, indicated by the blue colour in Fig. 11, as the features are normalized to have a mean of 0 and a standard deviation of 1. This normalization prevents any single feature from dominating the prediction, resulting in a more distributed contribution across multiple features. In addition, we incorporated regularization approaches, such as dropout, which could potentially diminish the influence of particular features, resulting in decreased SHAP values. Models often perform better with scaled features because scaling can lead to faster convergence and better numerical stability during training. Consequently, SHAP values derived from models trained on scaled data might be more consistent and reliable [59,60].

Nevertheless, when analyzing SHAP values derived from scaled features, it is crucial to bear in mind that they represent the influence of variations in the scaled (standardized) feature values, rather than the original feature values. The direction of the impact (positive or negative) remains consistent, but the magnitude needs to be interpreted in the context of the scaled features [61].

In the context of our research problem, this Fig. 11 aids in comprehending the features that influence the model's prediction towards the positive class (such as identifying a malicious event in IoT traffic) and the features that influence the prediction towards the negative class (such as benign traffic). Positive SHAP values, located on the right side of the x-axis, enhance the likelihood of the positive class, whereas negative SHAP values, found on the left side, diminish it. This interpretation facilitates the diagnosis and validation of the model's behaviour, ensuring that the model's decisions are made based on relevant features.

After identifying the top important features, we begin retraining the model using a reduced set of features from scratch, aiming to determine the optimal subset of features capable of achieving comparable performance to the original model. The gaps or differences between the bars in Fig. 6 guide the choice of the reduced feature set.

Subsequently, we compile a list of feature subsets and retrain and evaluate the model using the CIC-BCCC-NRC_IoT-2023 dataset and other datasets specified in Table 4. Through this evaluation process, we ascertain that a minimum of 7 features is necessary to achieve performance akin to the initial model's, as shown in Table 8.

The following section outlines the training history (Fig. 12) and confusion matrix (Fig. 13) for the CIC-BCCC-NRC_IoT-2023 dataset, along with confusion matrices for the datasets referenced in Table 4 are presented in Table 9.

Table 10 provides a detailed description of the top 7 features identified, which exhibit similar performance to the original model.

Finally, the performance comparison between the initial model and the reduced 7-feature set model is presented in Fig. 7.

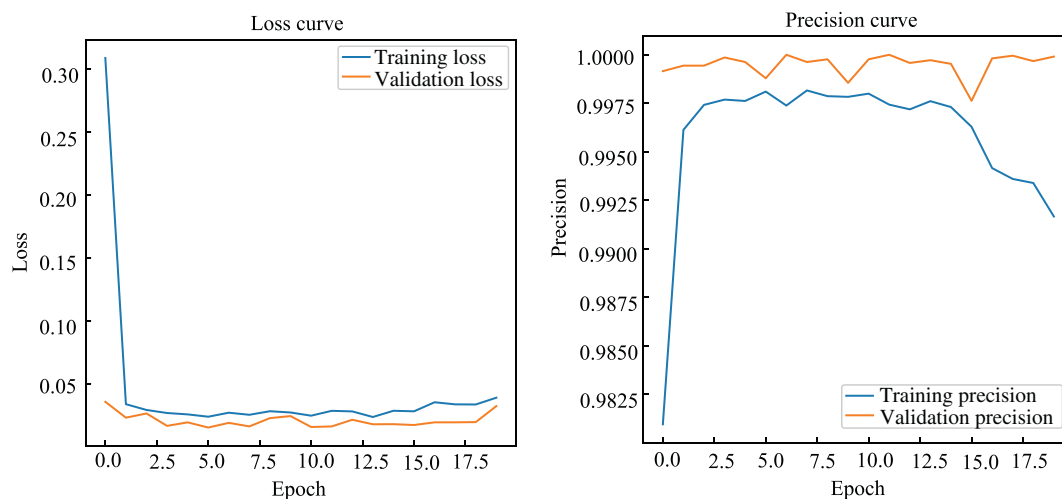


Fig. 12. Reduced set feature's training history results.

Table 9

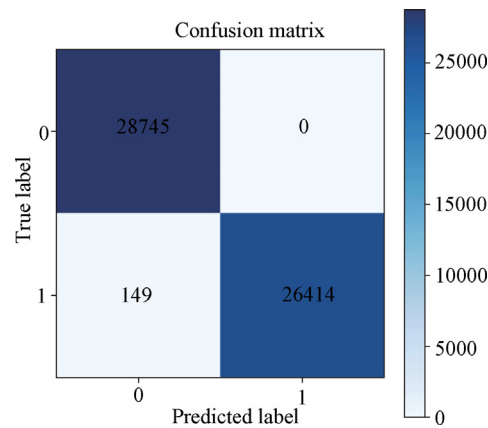
Confusion metrics for 7 features reduced model across all datasets.

Dataset	Accuracy	Precision	Recall	F1 score
CIC-BCCC-NRC_IoT-2023	0.9996	1	0.9944	0.9972
CIC-BCCC-NRC_ACI-IoT-2023	0.8842	0.881	0.9989	0.9363
CIC-BCCC-NRC_IoT-2022	0.9908	0.9914	0.993	0.9922
CIC-BCCC-NRC_Edge-IIoTset-2022	0.3330	0.333	1	0.4997
CIC-BCCC-NRC_IoT-HCRL-2019	0.9948	0.9956	0.9714	0.9834
CIC-BCCC-NRC_MQTTIoT-IDS-2020	0.9633	0.964	1	0.9817
CIC-BCCC-NRC_TONIoT-2021	0.9997	1	0.9999	0.9999
CIC-BCCC-NRC_UQ-IoT-2022	0.9711	0.9681	0.9999	0.9837
CIC-BCCC-NRC_IoMT-2024	0.9988	0.9989	0.9998	0.9993

Table 10

Top 7 best features for retraining.

Feature name	Description
CWR flag count	This represents the count of packets with the congestion window reduced (CWR) flag set. The CWR flag is set by the sending host to indicate that it received a TCP segment with the ECE flag set and had responded in a congestion control mechanism. Anomalies in the CWR flag count could indicate potential congestion or network attacks, such as congestion collapse or denial-of-service (DoS) attacks targeting the TCP congestion control mechanism.
ACK flag count	This denotes the count of packets with the acknowledgment (ACK) flag set. The ACK flag is used to acknowledge receipt of data packets. A sudden surge or drop in the ACK flag count could signify abnormal network behaviour, such as TCP connection hijacking attempts or ACK flooding attacks.
Bwd packets/s	This indicates the rate of backward (incoming) packets per second. Unusually high or low rates of backward packets might indicate malicious activities like network scanning or packet injection attacks.
Bwd bytes/bulk avg	This represents the average number of bytes transmitted in bulk in the backward direction. Significant deviations from the average bulk byte rate could suggest data exfiltration attempts or large-scale data transfer activities indicative of a security breach.
Bwd packet/bulk avg	This denotes the average number of packets transmitted in bulk in the backward direction. Deviations from the average bulk packet rate could indicate abnormal traffic patterns, such as bulk data transfers associated with malware propagation or botnet activity.
RST flag count	This indicates the count of packets with the reset (RST) flag set. The RST flag is used to reset a TCP connection. Anomalous spikes in the RST flag count may signal attempts to disrupt network communications, such as TCP reset attacks or connection teardown attempts by malware.
ECE flag count	This represents the count of packets with the explicit congestion notification (ECE) flag set. The ECE flag is used to indicate congestion on the network path. Abnormalities in the ECE flag count could suggest network congestion or targeted congestion-based attacks, such as congestion-aware DoS attacks or congestion control evasion techniques employed by attackers.

**Fig. 13.** Reduced set feature's confusion matrix results.

Additionally, [Table 11](#) illustrates the model size and training time for both the initial model and the reduced 7-feature set model. It is observed that there is a 45.77% decrease in model size, accompanied by a slight reduction in training time.

Due to the significant imbalance present in all datasets, characterized by a higher number of attack instances compared to benign ones, and considering that these datasets were used as test datasets without balancing, it is likely that this imbalance contributed to the high recall and precision observed in our model, as evidenced by the confusion matrices in [Tables 7](#) and [9](#). Nevertheless, given that the training was conducted on the balanced CIC-BCCC-NRC_IoT-2023 dataset, we are confident in the model's capability to accurately identify malicious instances. We anticipate that if the test datasets included a larger proportion of benign data, the performance difference would be negligible compared to the current results.

In summary, the better performance of the proposed self-attention-based 1D-CNN-LSTM approach for IoT attack detection can be attributed mostly to its robust feature extraction capabilities, efficient handling of high-dimensional data, and its capacity to highlight important data points via the self-attention mechanism. The integration of convolutional neural networks (CNNs) with long short-term memory (LSTM) networks allows the model to effectively capture spatial and temporal patterns in network data, hence facilitating accurate distinction between benign and malicious activities. Applying Shapley additive explanations (SHAP) to the

Table 11

Comparison of initial and reduced feature model sizes and training time.

	Total parameters	Trainable parameters	Non-trainable parameters	Training time
Initial model	251661 (983.05 KB)	251661 (983.05 KB)	0 (0.00 byte)	1198 s
Reduced 7 features model	136461 (533.05 KB)	136461 (533.05 KB)	0 (0.00 byte)	1067 s

model optimizes its performance by emphasizing the most significant characteristics, therefore simplifying the model and increasing computing efficiency without compromising accuracy. Moreover, the model's performance consistency across different enhanced IoT datasets emphasizes its resilience and flexibility, establishing it as a reliable and flexible solution for IoT security and intrusion detection.

6. Conclusion & future works

In this paper, a novel method called the self-attention-based 1D-CNN-LSTM, integrating CNNs, long short-term memory (LSTMs), and self-attention mechanisms, was developed to enhance the detection of IoT attacks within TCP network flows [62]. This approach capitalizes on the robust feature extraction capabilities of CNNs, the sequential data processing strengths of LSTMs, and the focused contextual insights provided by self-attention mechanisms, thereby establishing a sophisticated analytical framework.

Throughout this research, significant emphasis has been placed on the efficiency of the data preprocessing stage. We employ the CICFlowMeter tool for precise feature extraction from network traffic data and implement advanced machine-learning techniques for feature selection and model optimization. Using SHAP values to elucidate feature importance highlights our commitment to transparency and interpretability in machine learning models, which is essential in cybersecurity.

Furthermore, this research tested the initial and reduced models on nine augmented IoT datasets called CIC-BCCC-NRC_TabularIoTAttack-2024. The comparative results confirm that the proposed method performs better in detecting IoT attacks across various datasets, providing empirical evidence of the model's effectiveness in distinguishing malicious and benign samples within an IoT environment.

6.1. Future works

1. *Model architecture optimization:* Continuously refining the architecture of the self-attention-based 1D-CNN-LSTM mechanism can lead to better performance. This involves experimenting with different network depths, layer configurations, activation functions, and other architectural components to improve model expressiveness and learning capacity.
2. *Attention mechanism enhancement:* Further research into attention mechanisms tailored specifically for network flow data can lead to more effective models. This includes exploring different attention mechanisms such as multiplicative, multi-head, or hierarchical attention to capture different aspects of temporal dependencies and relationships within the data.
3. *Integration of external context:* Incorporating additional contextual information, such as network topology, device metadata, or domain-specific knowledge, into the model architecture can improve its ability to interpret network flow data accurately. This can enhance the model's understanding of the underlying network dynamics and improve its predictive performance.
4. *Data preprocessing and feature engineering:* Developing more sophisticated preprocessing techniques and feature engineering strategies specific to network flow data can help extract more meaningful representations from the raw data. This may involve incorporating domain knowledge to extract relevant features, handling missing or noisy data, or applying dimensionality reduction techniques to reduce computational complexity.
5. *Model improvement using federated learning approach:* Enhancing the model architecture to operate within a federated framework facilitates the utilization of locally updated models, enabling the training of attack detection algorithms on IoT devices with similar characteristics. This approach yields more refined and precise models tailored to specific categories of IoT devices, consequently enhancing overall model performance and efficacy.

6.2. Availability

After publishing this paper, the augmented datasets, namely CIC-BCCC-NRC_TabularIoTAttack-2024, are available on the collaborator universities' websites.

CRedit authorship contribution statement

Tinshu Sasi: Writing – review & editing, Writing – original draft, Methodology. **Arash Habibi Lashkari:** Supervision, Methodology. **Rongxing Lu:** Writing – review & editing, Validation. **Pulei Xiong:** Writing – review & editing, Validation. **Shahreaz Iqbal:** Writing – review & editing, Validation.

Acknowledgments

This project was supported in part by collaborative research funding from the National Research Council of Canada's Artificial Intelligence for Logistics Program.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. List of all malicious activities present in all augmented datasets

This appendix provides a comprehensive list of all malicious activities identified in the datasets referenced in Table 4. Each activity is accompanied by a brief description to aid in understanding the various threats documented.

Table A

List of all malicious activities present in all datasets in Table 4 [4,63].

Malicious activity	Description
Benign traffic	Normal network traffic with no malicious intent.
DDoS ACK fragmentation	Distributed Denial of Service attack using fragmented ACK packets to overwhelm the target.
DDoS ICMP flood	DDoS attacks send many ICMP packets to overload the target.
DDoS ICMP fragmentation	Fragmented ICMP packets are used to consume the target's resources in a DDoS attack.
DDoS PSN ACK flood	Flooding the target with TCP packets with PSN and ACK flags set to disrupt services.
DDoS RST FIN flood	Using TCP RST and FIN flags to flood and disrupt target systems.
DDoS SYN flood	Overwhelming a target with TCP SYN packets to exhaust resources and deny service.
DDoS fynonymous IP flood	Using multiple IP addresses to send DDoS traffic to the target.
DDoS TCP flood	Flooding the target with TCP packets to cause a denial of service.
DDoS UDP flood	Sending large UDP packets to overwhelm the target's resources.
DDoS UDP fragmentation	Fragmented UDP packets are used in a DDoS attack to disrupt services.
DNS spoofing	Manipulating DNS responses to redirect traffic to malicious sites.
DoS HTTP flood	Denial of Service attack using HTTP requests to overwhelm a web server.
DoS SYN flood	Sending numerous TCP SYN packets to cause a denial of service on the target.
DoS TCP flood	Flooding the target with TCP packets to cause a denial of service.
DoS UDP flood	Overwhelming the target with UDP packets to cause a denial of service.
MITM ARP spoofing	Man-in-the-middle attack using ARP spoofing to intercept network traffic.
Mirai GRE-ETH flood	Mirai botnet attack using GRE over Ethernet packets to flood the target.
Mirai GRE-IP flood	Mirai botnet attack using GRE over IP packets to flood the target.
Mirai UDP plain	Mirai botnet attack using plain UDP packets to overwhelm the target.
Dictionary brute force	Attempting to gain access by trying numerous passwords from a predefined list.
DoS DNS flood	Overwhelming a DNS server with requests to cause a denial of service.
Recon host discovery	Scanning the network to discover active hosts.
Recon OS scan	Scanning to identify the operating systems of hosts on the network.
Recon ping sweep	Sending ICMP Echo requests to multiple hosts to determine their status.
Recon port scan	Scanning a host to discover open ports and services.
Recon vulnerability scan	Scanning systems to identify potential vulnerabilities.
Backdoor	Creating or exploiting a hidden access point into a system.
DDoS HTTP flood	Overwhelming a web server with HTTP requests in a DDoS attack.
DDoS TCP SYN flood	A specific type of SYN flood used in a DDoS attack to exhaust resources.
OS fingerprinting	Identifying the operating system of a target by analyzing packet signatures.
Password attack	Attempting to gain unauthorized access by guessing or cracking passwords.
Port scanning	Identifying open ports and services on a networked device.
Ransomware	Malicious software that encrypts files and demands ransom for decryption.
SQL injection	Inserting malicious SQL queries into input fields to manipulate databases.
Uploading attacks	Uploading malicious files to a server to exploit vulnerabilities.
Vulnerability scanner	Automated tool to scan systems for known vulnerabilities.
XSS	Cross-site scripting attack to inject malicious scripts into web pages.
DoS ICMP flood	Overloading a target with ICMP Echo requests to cause a denial of service.
MQTT DDoS connect flood	Using the MQTT protocol to send a large number of connection requests in a DDoS attack.
MQTT DDoS publish flood	Flooding the target with MQTT publish messages to cause a denial of service.
MQTT DoS connect flood	Using MQTT connect messages to cause a denial of service.
MQTT DoS publish flood	Using MQTT publish messages to cause a denial of service.
MQTT malformed	Sending malformed MQTT packets to disrupt the target.
Mirai ACK flood	Mirai botnet attack using ACK packets to overwhelm the target.
Mirai HTTP flood	Using the Mirai botnet to flood a web server with HTTP requests.
Mirai host brute force	Mirai botnet are attempting to brute force login credentials on targets.
Mirai UDP flood	Mirai botnet sends a large volume of UDP packets to overwhelm the target.
Scan host port	Scanning hosts to identify open ports and services.
Scan port OS	Scanning ports to identify the target's operating system.
MQTT brute force	Attempting to gain unauthorized access by brute forcing MQTT credentials.
Scan aggressive	Comprehensive scanning to gather detailed information about the target.
Scan UDP attack	Scanning using UDP packets to identify open ports and services.
Sparta SSH brute force	Using the Sparta tool to brute force SSH login credentials.
DDoS	Distributed Denial of Service attack using multiple sources to overwhelm the target.
DoS	Denial of Service attack aimed at making a service unavailable.
Injection	Attacks that involve injecting malicious input into an application.
MITM	Man-in-the-middle attack where the attacker intercepts communication between two parties.
ACK flood	Flooding the target with TCP ACK packets to disrupt services.

(continued on next page)

Table A (continued)

Malicious activity	Description
HTTP flood	Overwhelming a web server with HTTP requests to cause a denial of service.
Recon service detection	Scanning to detect active services on a network.
SYN flood	Flooding a target with SYN packets to exhaust resources and deny service.
Telnet brute force	Brute forcing Telnet login credentials to gain unauthorized access.
UDP flood	Overwhelming a target with UDP packets to cause a denial of service.

Appendix B. CICFlowMeter extracted features list

This appendix presents a comprehensive list of the features extracted by the CICFlowMeter tool. Each feature is described briefly to help understand the various aspects of network traffic that were captured and analyzed.

Table B

CICFlowMeter extracted features list.

Feature name	Description
Flow ID	Flow identifier
Src IP	Source IP
Src port	Source port
Dst IP	Destination IP
Dst port	Destination port
Flow duration	Duration of the flow in microseconds
Total Fwd packet	Total packets in the forward direction
Total Bwd packets	Total packets in the backward direction
Total length of fwd packet	Total size of packet in forward direction
Total length of bwd packet	Total size of packet in backward direction
Fwd packet length min	Minimum size of the packet in forward direction
Fwd packet length max	Maximum size of the packet in forward direction
Fwd packet length mean	Mean size of packet in forward direction
Fwd packet length std	Standard deviation size of packet in forward direction
Bwd packet length min	Minimum size of packet in backward direction
Bwd packet length max	Maximum size of the packet in backward direction
Bwd packet length mean	Mean size of the packet in backward direction
Bwd packet length std	Standard deviation size of packet in backward direction
Flow bytes/s	Number of flow bytes per second
Flow packets/s	Number of flow packets per second
Flow IAT mean	Mean time between two packets sent in the flow
Flow IAT Std	Standard deviation time between two packets sent in the flow
Flow IAT max	Maximum time between two packets sent in the flow
Flow IAT min	Minimum time between two packets sent in the flow
Fwd IAT min	Minimum time between two packets sent in the forward direction
Fwd IAT max	Maximum time between two packets sent in the forward direction
Fwd IAT mean	Mean time between two packets sent in the forward direction
Fwd IAT Std	Standard deviation time between two packets sent in the forward direction
Fwd IAT total	Total time between two packets sent in the forward direction
Bwd IAT min	Minimum time between two packets sent in the backward direction
Bwd IAT max	Maximum time between two packets sent in the backward direction
Bwd IAT mean	Mean time between two packets sent in the backward direction
Bwd IAT Std	Standard deviation time between two packets sent in the backward direction
Bwd IAT total	Total time between two packets sent in the backward direction
Fwd PSH flags	Number of times the PSH flag was set in packets travelling in the forward direction (0 for UDP)
Bwd PSH flags	Number of times the PSH flag was set in packets travelling in the backward direction (0 for UDP)
Fwd URG flags	Number of times the URG flag was set in packets travelling in the forward direction (0 for UDP)
Bwd URG flags	Number of times the URG flag was set in packets travelling in the backward direction (0 for UDP)
Fwd header length	Total bytes used for headers in the forward direction
Bwd header length	Total bytes used for headers in the backward direction
FWD packets/s	Number of forward packets per second
Bwd packets/s	Number of backward packets per second
Packet length min	Minimum length of a packet
Packet length max	Maximum length of a packet
Packet length mean	Mean length of a packet
Packet length Std	Standard deviation length of a packet
Packet length variance	Variance length of a packet
FIN flag count	Number of packets with FIN
SYN flag count	Number of packets with SYN
RST flag count	Number of packets with RST
PSH flag count	Number of packets with PUSH
ACK flag count	Number of packets with ACK

(continued on next page)

Table B (continued)

Feature name	Description
URG flag count	Number of packets with URG
CWR flag count	Number of packets with CWR
ECE flag count	Number of packets with ECE
Down/up ratio	Download and upload ratio
Average packet size	Average size of packet
Fwd segment size avg	Average size observed in the forward direction
Bwd segment size avg	Average size observed in the backward direction
Fwd bytes/bulk avg	Average number of bytes bulk rate in the forward direction
Fwd packet/bulk avg	Average number of packets bulk rate in the forward direction
Fwd bulk rate avg	Average number of bulk rate in the forward direction
Bwd bytes/bulk avg	Average number of bytes bulk rate in the backward direction
Bwd packet/bulk avg	Average number of packets bulk rate in the backward direction
Bwd bulk rate avg	Average number of bulk rate in the backward direction
Subflow fwd packets	The average number of packets in a sub-flow in the forward direction
Subflow fwd bytes	The average number of bytes in a sub-flow in the forward direction
Subflow bwd packets	The average number of packets in a sub-flow in the backward direction
Subflow bwd bytes	The average number of bytes in a sub-flow in the backward direction
Fwd init win bytes	The total number of bytes sent in the initial window in the forward direction
Bwd init win bytes	The total number of bytes sent in the initial window in the backward direction
Fwd act data pkts	Count of packets with at least 1 byte of TCP data payload in the forward direction
Fwd seg size min	Minimum segment size observed in the forward direction
Active min	Minimum time a flow was active before becoming idle
Active mean	Mean time a flow was active before becoming idle
Active max	Maximum time a flow was active before becoming idle
Active std	Standard deviation time a flow was active before becoming idle
Idle min	Minimum time a flow was idle before becoming active
Idle mean	Mean time a flow was idle before becoming active
Idle max	Maximum time a flow was idle before becoming active
Idle std	Standard deviation time a flow was idle before becoming active

References

- [1] L. Vu, Q.U. Nguyen, D.N. Nguyen, D.T. Hoang, E. Dutkiewicz, Deep transfer learning for IoT attack detection, *IEEE Access* 8 (2020) 107335–107344.
- [2] Z. Wang, J. Li, S. Yang, X. Luo, D. Li, S. Mahmoodi, A lightweight IoT intrusion detection model based on improved BERT-of-Theseus, *Expert Systems with Applications* 238 (2024) 122045.
- [3] Z. Wang, D. Liu, Y. Sun, X. Pang, P. Sun, F. Lin, J.C.S. Lui, K. Ren, A survey on IoT-enabled home automation systems: Attacks and defenses, *IEEE Communications Surveys & Tutorials* 24 (4) (2022) 2292–2328.
- [4] T. Sasi, A.H. Lashkari, R. Lu, P. Xiong, S. Iqbal, A comprehensive survey on IoT attacks: Taxonomy, detection mechanisms and challenges, *Journal of Information and Intelligence* 2 (6) (2023) 455–513.
- [5] A. Simmons, Internet of Things (IoT) architecture: Layers explained [06-04 -2023], <https://dgtlinfra.com/internet-of-things-iot-architecture/>, November 2022.
- [6] J. Sengupta, S. Ruj, S.D. Bit, A comprehensive survey on attacks, security issues and blockchain solutions for IoT and IIoT, *Journal of Network and Computer Applications* 149 (1) (2020) 102481.
- [7] O. Oladehin, D. Griffin, C. Vieru, B. Francis, C. Williams, P. Fitzsimons, H. Rajagopalan, R. Dsouza, A. Bhaskar, D. Walters, et al., IoT Lens [06-04-2023], <https://docs.aws.amazon.com/wellarchitected/latest/iot-lens/edge-layer.html>, March 2023.
- [8] L. Bellehumeur, The 5 layers of IoT architecture that give it super powers [06-04-2023], <https://novotech.com/blogs/news/the-5-layers-of-iot-architecture-that-give-it-super-power>, January 2023.
- [9] R. Agar, IoT architecture guide. Major and additional layers of IoT system [06-04-2023], <https://www.helpwire.app/blog/iot-architecture/>, November 2022.
- [10] A. Arampatzis, Top 10 vulnerabilities that make IoT devices insecure [06-04-2023], <https://venafi.com/blog/top-10-vulnerabilities-make-iot-devices-insecure/>, October 2022.
- [11] L. Sun, Q. Du, A review of physical layer security techniques for Internet of Things: challenges and solutions, *Entropy* 20 (10) (2018) 730.
- [12] M.M. Ogonji, G. Okeyo, J.M. Wafula, A survey on privacy and security of Internet of Things, *Computer Science Review* 38 (2020) 100312.
- [13] B.B. Zarpelao, R.S. Miani, C.T. Kawakani, S.C. de Alvarenga, A survey of intrusion detection in Internet of Things, *Journal of Network and Computer Applications* 84 (2017) 25–37.
- [14] S. Hajiheidari, K. Wakil, M. Badri, N.J. Navimipour, Intrusion detection systems in the Internet of Things: A comprehensive investigation, *Computer Networks* 160 (4) (2019) 165–191.
- [15] D.E. Kouicem, A. Bouabdallah, H. Lakhlef, Internet of Things security: A top-down survey, *Computer Networks* 141 (4) (2018) 199–221.
- [16] G. Fersi, Fog computing and Internet of Things in one building block: A survey and an overview of interacting technologies, *Cluster Computing* 24 (2021) 2757–2787.
- [17] N.K. Tran, M. Ali Babar, J. Boan, Integrating blockchain and Internet of Things systems: A systematic review on objectives and designs, *Journal of Network and Computer Applications* 173 (1) (2021) 102844.
- [18] R.A. Memon, J.P. Li, J. Ahmed, M.I. Nazeer, M. Ismail, K. Ali, Cloud-based vs. blockchain-based IoT: A comparative survey and way forward, *Frontiers of Information Technology & Electronic Engineering* 21 (4) (2020) 563–586.
- [19] P.J. Taylor, T. Dargahi, A. Dehghantanha, R.M. Parizi, K.-K.R. Choo, A systematic literature review of blockchain cyber security, *Digital Communications and Networks* 6 (2) (2020) 147–156.
- [20] R.R. Krishna, A. Priyadarshini, A.V. Jha, B. Appasani, A. Srinivasulu, N. Bizon, State-of-the-art review on IoT threats and attacks: Taxonomy, challenges and solutions, *Sustainability* 13 (16) (2021) 9463.
- [21] L. Cui, S. Yang, F. Chen, Z. Ming, N. Lu, J. Qin, A survey on application of machine learning for Internet of Things, *International Journal of Machine Learning and Cybernetics* 9 (8) (2018) 1399–1417.
- [22] L. Xiao, Y. Li, G. Han, G. Liu, W. Zhuang, PHY-layer spoofing detection with reinforcement learning in wireless networks, *IEEE Transactions on Vehicular Technology* 65 (12) (2016) 10037–10047.
- [23] S.M. Tahsien, H. Karimipour, P. Spachos, Machine learning based solutions for security of Internet of Things (IoT): A survey, *Journal of Network and Computer Applications* 161 (1) (2020) 102630.
- [24] A. Chatterjee, B.S. Ahmed, IoT anomaly detection methods and applications: A survey, *Internet of Things* 19 (2022) 100568.

- [25] P. Manirih, A.N. Mahmood, M.J.M. Chowdhury, A study on malicious software behaviour analysis and detection techniques: Taxonomy, current trends and challenges, *Future Generation Computer Systems* 130 (2022) 1–18.
- [26] M.F. Razali, M.N. Razali, F.Z. Mansor, G. Muruti, N. Jamil, IoT honeypot: A review from researcher's perspective, in: *Proceedings of the 2018 IEEE Conference on Application, Information and Network Security (AINS)*, IEEE, Malaysia, 2018, pp. 93–98.
- [27] IBM, What is security information and event management (SIEM)? [06-04-2023], <https://www.ibm.com/topics/siem>, October 2023.
- [28] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIOT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (2023) 5941.
- [29] I.A.N. Authority, Protocol numbers [06-04-2023], <https://www.iana.org/assignments/protocol-numbers/protocol-numbers.xhtml>.
- [30] A.H. Lashkari, A. Seo, G.D. Gil, A.A. Ghorbani, CICFlowMeter (formerly ISCXFlowMeter) [06-04-2023], <https://www.unb.ca/cic/research/applications.html>, January 2022.
- [31] A. Abulkhair, Data imputation demystified | Time series data [06-04-2023], <https://medium.com/@aaabulkhair/data-imputation-demystified-time-series-data-69bc9c798cb7>, June 2023.
- [32] Bhanupsingh, Handling missing data with KNN imputer [06-04-2023], <https://medium.com/@bhanupsingh484/handling-missing-data-with-knn-imputer-927d49b09015>, August 2023.
- [33] A.W. Services, What is hyperparameter tuning? [06-04-2023], <https://aws.amazon.com/what-is/hyperparameter-tuning/>, August 2023.
- [34] Scikit-learn.org, Train_test_split, [06-04-2023]. https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.train_test_split.html.
- [35] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 4768–4777.
- [36] C. Molnar, Interpretable machine learning: A guide for making black box models explainable [06-04-2023], <https://christophm.github.io/interpretable-ml-book/>, August 2023.
- [37] N. Bastian, D. Bierbrauer, M. McKenzie, E. Nack, ACI IoT network traffic dataset 2023, *IEEE Dataport* (2023), <https://doi.org/10.21227/qacj-3x32> [06-04-2023].
- [38] S. Dadkhah, H. Mahdikhani, P.K. Danso, A. Zohourian, K.A. Truong, A.A. Ghorbani, Towards the development of a realistic multidimensional IoT profiling dataset, in: *2022 19th Annual International Conference on Privacy, Security & Trust (PST)*, IEEE, Fredericton, NB, Canada, August 2022, pp. 1–11.
- [39] S. Dadkhah, E.C.P. Neto, R. Ferreira, R.C. Molokwu, S. Sadeghi, A. Ghorbani, CICIOT2024: Attack vectors in healthcare devices-a multiprotocol dataset for assessing IoT device security, *Internet of Things* 28 (2024) 101351.
- [40] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications: Centralized and federated learning [06-04-2023], <https://dx.doi.org/10.21227/mbc1-1h68>, January 2022.
- [41] H. Kang, D.H. Ahn, G.M. Lee, J.D. Yoo, K.H. Park, H.K. Kim, IoT network intrusion dataset [06-04-2023], <https://dx.doi.org/10.21227/q70p-q449>, September 2019.
- [42] H. Hindy, C. Tachtatzis, R. Atkinson, E. Bayne, X. Bellekens, MQTT-IoT-IDS2020: MQTT Internet of Things intrusion detection dataset. <https://dx.doi.org/10.21227/bhxy-ep04>, June 2020.
- [43] K. He, K. Dan, Z. Zhang, G. Mengmeng, J. Yu, UQ IoT IDS dataset 2021 [06-04-2023], <https://espace.library.uq.edu.au/view/UQ:17b44bb>, 2022.
- [44] Radware, Fragmented ACK attack, [04-15-2024]. <https://www.radware.com/security/ddos-knowledge-center/ddospedia/fragmented-ack-attack/>.
- [45] Akamai, What is an ICMP flood DDoS attack?, [04-15-2024]. <https://www.akamai.com/glossary/what-is-icmp-flood-ddos-attack>.
- [46] NETSCOUT, IP/ICMP fragmentation attacks, [04-15-2024]. <https://www.netscout.com/what-is-ddos/ip-icmp-fragmentation>.
- [47] D. Guard, ACK and PUSH ACK flood, [04-15-2024]. <https://ddos-guard.net/en/terms/ddos-attack-types/ack-push-ack-flood>.
- [48] D. Guard, RST and FIN flood, [04-15-2024]. <https://ddos-guard.net/en/terms/ddos-attack-types/rst-fin-flood>.
- [49] Cloudflare, SYN flood attack, [04-15-2024]. <https://www.cloudflare.com/learning/ddos/syn-flood-ddos-attack/>.
- [50] D. Guard, Synonymous IP attack, [04-15-2024]. <https://ddos-guard.net/terms/ddos-attack-types/synonymous-ip-attack>.
- [51] Imperva, TCP SYN flood, [04-15-2024]. <https://www.imperva.com/learn/ddos/syn-flood/>.
- [52] Imperva, UDP flood, [04-15-2024]. <https://www.imperva.com/learn/ddos/udp-flood/>.
- [53] D. Guard, Fragmented UDP flood, [04-15-2024]. <https://ddos-guard.net/en/terms/ddos-attack-types/udp-fragmentation-flood>.
- [54] P. Security, What is DNS spoofing? + 5 tips to prevent it [04-15-2024], <https://www.pandasecurity.com/en/mediacenter/dns-spoofing/>, Mar 2024.
- [55] Imperva, HTTP flood, [04-15-2024]. <https://www.imperva.com/learn/ddos/http-flood/>.
- [56] Radware, What is the Mirai botnet?, [04-15-2024]. <https://www.radware.com/security/ddos-knowledge-center/ddospedia/mirai/>.
- [57] R. Winward, IoT attack handbook a field guide to understanding IoT attacks from the Mirai botnet to its modern variants, [04-15-2024]. <https://www.radware.com/getattachment/402db7f3-0467-4fa3-bb9a-ae88b728e91b/MiraiHandbookEbookFinal04.pdf> f.aspx.
- [58] Imperva, ARP spoofing, [04-15-2024]. <https://www.imperva.com/learn/application-security/arp-spoofing/>.
- [59] A. Bhandari, Feature scaling: Engineering, normalization, and standardization [07-12-2023], <https://www.analyticsvidhya.com/blog/2020/04/feature-scaling-machine-learning-normalization-standardization/>, June 2024.
- [60] H. Wang, Q. Liang, J.T. Hancock, T.M. Khoshgoftaar, Feature selection strategies: A comparative analysis of SHAP-value and importance-based methods, *Journal of Big Data* 11 (44) (2024), <https://doi.org/10.1186/s40537-024-00905-w>.
- [61] K. Aas, M. Jullum, A. Løland, Explaining individual predictions when features are dependent: More accurate approximations to Shapley values, *Artificial Intelligence* 298 (2021) 103502.
- [62] S. Yaras, M. Dener, IoT-based intrusion detection system using new hybrid deep learning algorithm, *Electronics* 13 (6) (2024) 1053.
- [63] P. Victor, A.H. Lashkari, R. Lu, T. Sasi, P. Xiong, S. Iqbal, IoT malware: An attribute-based taxonomy, detection mechanisms and challenges, *Peer-to-Peer Networking and Applications* 16 (2023) 1380–1431.



Tinshu Sasi is a Master's student at the University of New Brunswick's (UNB) Faculty of Computer Science in Canada. In 2014, he earned a bachelor's degree in computer science from Manav Rachna International University in India. He also has over 8 years of experience working as a Senior Software Engineer developing enterprise-level software solutions. Cybersecurity, the Internet of Things, machine learning, and deep learning are among his key research interests.



Arash Habibi Lashkari is a Canada Research Chair (CRC) in Cybersecurity. He is a Senior member of IEEE and an Associate Professor at York University. Before this, he was an Associate Professor at the Faculty of Computer Science, University of New Brunswick (UNB), and the Research Coordinator of the Canadian Institute for Cybersecurity (CIC). His research focuses on cyber threat modelling and detection, malware analysis, big data security, internet traffic analysis, and cybersecurity dataset generation. Dr. Lashkari has over 25 years of teaching experience spanning several international universities and was responsible for designing the first cybersecurity Capture the Flag (CTF) competition for post-secondary students in Canada. He has been the recipient of 15 awards at international computer security competitions, including three gold awards, and was recognized as one of Canada's Top 150 Researchers for 2017. In 2020, Dr. Lashkari was recognized with the University of New Brunswick's prestigious Teaching Innovation Award for his personally-created teaching methodology, the Think-Que-Cussion Method. He is the author of ten published books and more than 110 academic articles on a variety of cybersecurity-related topics and the co-author of the national award-winning article series "Understanding Canadian Cybersecurity Laws", which was recently recognized with a Gold Medal at the 2020 Canadian Online Publishing Awards, remotely held in 2021.



Rongxing Lu is a University Research Scholar and Professor at the University of New Brunswick's (UNB) Faculty of Computer Science (FCS). From April 2013 until August 2016, he was an Assistant Professor at Nanyang Technological University's (NTU) School of Electrical and Electronic Engineering in Singapore. From May 2012 until April 2013, Rongxing Lu was a Postdoctoral Fellow at the University of Waterloo. He received the most prestigious "Governor General's Gold Medal" when he received his Ph.D. degree from the Department of Electrical & Computer Engineering, University of Waterloo, Canada, in 2012; and in 2013, he won the 8th IEEE Communications Society (ComSoc) Asia Pacific (AP) Outstanding Young Researcher Award. Dr. Lu is a member of the IEEE. His research interests include applied cryptography, privacy-enhancing technologies, and the security and privacy of IoT-big data. Dr. Lu now holds many positions, including the Mastercard IoT Research Chair, the Chair of IEEE ComSoc CIS-TC (Communications and Information Security Technical Committee), and the founding Co-chair of IEEE TEMS Blockchain and Distributed Ledgers Technologies Technical Committee (BDLT-TC).



Pulei Xiong is a Research Officer in the Cybersecurity team at the National Research Council Canada. His current research interests focus on the study of robust machine learning, encompassing offensive, defensive, and assessment methods for developing and deploying machine learning systems that are resilient against adversarial attacks throughout the entire machine learning pipeline. Currently, he is the Principal Investigator (PI) leading several research projects in this area, collaborating with experts from academia and industry. In addition to his work in robust machine learning, his research interests include privacy-preserving technologies and their application, and security compliance for emerging technologies. Prior to joining NRC, Pulei gained extensive experience in the cybersecurity industry. He is widely recognized as a cybersecurity consultant, renowned for his leadership in developing the Protection Profile for Mobile Devices, which stands as the first industrial security standard of its kind for mobile computing. Pulei has held the certification of GIAC Mobile Device Security Analyst (GMOB) since 2016.



Shahrear Iqbal is a Research officer at the National Research Council (NRC) Canada. He received his Ph.D. in Cybersecurity from Queen's University, Kingston, ON, Canada 2017. Before joining NRC, he was a researcher at Security Compass, Toronto. He is a specialist in software security in the areas of mobile, IoT, and connected devices. Previously, he was a faculty member in the Department of Computer Science and Engineering (CSE) at Bangladesh University of Engineering and Technology (BUET). He is currently doing research on the security of connected devices with a focus on context-aware authentication and automated defence mechanism.